

top

КОМПЬЮТЕРНАЯ
АКАДЕМИЯ

{ПРОГРАММИРОВАНИЕ}

ПАТТЕРНЫ

ПРОЕКТИРОВАНИЯ

ПОРОЖДАЮЩИЕ
ПАТТЕРНЫ

ПРОЕКТИРОВАНИЯ

СТРУКТУРНЫЕ

ПАТТЕРНЫ

ПАТТЕРНЫ

ПОВЕДЕНИЯ

Содержание

Урок №1.

Порождающие паттерны проектирования

1. Введение в паттерны проектирования	6
1.1. О тенденциях в развитии паттернов в программировании.....	7
1.2. Причины возникновения паттернов проектирования.....	8
1.3. Понятие паттерна проектирования.....	9
1.4. Практическое применение паттернов проектирования.....	11
1.5. Классификация паттернов.....	14
2. Краткое введение в UML	17
2.1. Диаграмма классов	17
2.2. Диаграмма объектов.....	23
3. Порождающие паттерны	25
3.1. Abstract Factory	25
3.2. Builder.....	38
3.3. Factory Method	49
3.4. Prototype	63
3.5. Singleton	72
3.6. Анализ и сравнение порождающих паттернов.....	79
4. Домашнее задание	81
5. Используемые информационные источники.....	82

Урок №2.

Структурные паттерны

1. Понятие структурного паттерна	84
2. Паттерн Adapter	86

2.1. Цель паттерна.....	86
2.2. Причина возникновения паттерна	86
2.3. Структура паттерна	87
2.4. Результаты использования паттерна.....	90
2.5. Практический пример использования паттерна.....	91
3. Паттерн Bridge.....	94
3.1. Цель паттерна.....	94
3.2. Причины возникновения паттерна	94
3.3. Структура паттерна	95
3.4. Результаты использования паттерна.....	96
3.5. Практический пример использования паттерна.....	97
4. Паттерн Composite.....	99
4.1. Цель паттерна.....	99
4.2. Причины возникновения паттерна	99
4.3. Структура паттерна	100
4.4. Результаты использования паттерна.....	101
4.5. Практический пример использования паттерна.....	102
5. Паттерн Decorator.....	105
5.1. Цель паттерна.....	105
5.2. Причины возникновения паттерна	105
5.3. Структура паттерна	106
5.4. Результаты использования паттерна.....	108
5.5. Практический пример использования паттерна.....	109
6. Паттерн Facade	114
6.1. Цель паттерна.....	115
6.2. Причины возникновения паттерна	115
6.3. Структура паттерна	116
6.4. Результаты использования паттерна.....	118
6.5. Практический пример	118
7. Паттерн Flyweight	121

7.1. Цель паттерна.....	121
7.2. Причины возникновения паттерна.....	121
7.3. Структура паттерна.....	122
7.4. Результаты использования паттерна.....	124
7.5. Практический пример	125
8. Паттерн Proxy	127
8.1. Цель паттерна.....	127
8.2. Причины возникновения паттерна.....	128
8.3. Структура паттерна.....	129
8.4. Результаты использования паттерна.....	130
8.5. Практический пример	131
9. Анализ и сравнение структурных паттернов	133
10. Домашнее задание	136

Урок № 3.

Паттерны поведения

1. Понятие паттерна поведения.....	138
2. Паттерн Chain Of Responsibility.....	139
3. Паттерн Command.....	141
4. Паттерн State.....	143
5. Паттерн Template Method	144
6. Паттерн Mediator	146
7. Экзаменационное задание	148

Материалы урока прикреплены к данному PDF-файлу. Для доступа к материалам, урок необходимо открыть в программе Adobe Acrobat Reader.

Урок №1.
**Порождающие паттерны
проектирования**

1. Введение в паттерны проектирования

Термин «паттерн» (*pattern*) следует понимать, как «образец». Часто его заменяют термином «шаблон» (*template*). По словам Кристофера Александра¹, «... любой паттерн описывает задачу, которая снова и снова возникает в нашей работе, а также принцип её решения, причем таким образом, что решение можно использовать миллион раз, ничего не изобретая заново...» [GoF95]. Такое определение паттерна существует в архитектуре (т.е. строительстве), но оно очень подходит и для определения паттерна в программировании.

В настоящем уроке мы рассмотрим основные понятия, связанные с паттернами проектирования в объектно-ориентированном программировании, также сделаем вступление в унифицированный язык визуального моделирования UML (*Unified Modeling Language*), с использованием которого описывается структура всех паттернов проектирования. Главной задачей урока является изложение важной категории паттернов объектно-ориентированного проектирования — порождающих паттернов (*Creational Patterns*), изначально описанных в [GoF95]. Все примеры практического использования паттернов представлены на языке C#, их исходные коды прилагаются к уроку.

¹ Кристофер Александр — архитектор, составивший в 1970-х годах XX ст. каталог паттернов проектирования в области строительной архитектуры. Позже его идея получила широкое развитие в области разработки программного обеспечения

1.1. О тенденциях в развитии паттернов в программировании

Идея паттернов проектирования первоначально возникла в архитектуре. Архитектор Кристофер Александр написал две революционные книги, содержащие описание шаблонов в строительной архитектуре и городском планировании: «*A Pattern Language: Towns, Buildings, Contributions*» (1977 г.), «*The Timeless Way of Building*» (1979 г.). В этих книгах были представлены общие идеи, которые могли использоваться даже в областях, не имеющих отношения к архитектуре, в том числе и в программировании.

В 1987 году Кент Бэк (*Kent Beck*) и Вард Каннигем (*Ward Cunningham*) на основе идеи Кристофера Александра разработали шаблоны разработки программного обеспечения для графических оболочек на языке Smalltalk. В 1988 году Эрих Гамма (*Erich Gamma*) начал писать докторскую диссертацию при цюрихском университете об общей переносимости методики паттернов проектирования на разработку программного обеспечения. В 1989–1991 годах Джеймс Коплин (*James Coplien*) трудился над разработкой идиом для программирования на C++ и опубликовал в 1991 году книгу «*Advanced C++ Idioms*». В этом же году Эрих Гамма заканчивает свою докторскую диссертацию и переезжает в США, где в сотрудничестве с Ричардом Хелмом (*Richard Helm*), Ральфом Джонсоном (*Ralph Johnson*) и Джоном Влиссидсом (*John Vlissides*) публикует книгу «*Design Patterns — Elements of Reusable Object-Oriented Software*» (русскоязычный перевод книги — [GoF95]). В этой книге описаны 23 паттерна проектирования. Также команда авторов этой книги известна под названием «Банда четырёх» (*Gang*

of Four, часто сокращается — GoF). Эта книга и стала причиной роста популярности паттернов проектирования.

Таким образом, широкое использование паттернов в программировании началось из описания базовых 23-х шаблонов проектирования «Банды четырех».

Следующим шагом стало описание Мартином Фаулером «*Enterprise Patterns*», где были раскрыты типичные решения при разработке корпоративных приложений, например, работа с базами данных, транзакциями и т.п.

Джошуа Криевски показал, как можно постоянным рефакторингом¹, руководствуясь базовыми принципами ООП, обеспечить эволюцию кода, перемещая его от одного паттерна к другому в зависимости от требований.

После начала акцентирования внимания на модульном тестировании (*Unit Testing*) появилось понятие тестируемости программного кода. Все паттерны при этом были переосмыслены с позиций тестируемости. При этом, например, оказалось, что паттерн **Singleton** — это анти-паттерн (см. пункт 3.5), а **Abstract Factory** (см. пункт 3.1) вообще заменили **IoC** (*Inversion of Control*) контейнеры. После выхода книги «*xUnit Test Patterns*» в 2008 году появилось несколько десятков паттернов тестирования.

1.2. Причины возникновения паттернов проектирования

В конце 80-х годов XX века в сфере разработки программного обеспечения, в частности, объектно-ориентированном проектировании, накопилось много различных

¹ Рефакторинг — изменения внутреннего устройства программного кода без модификации его внешней функциональности

сходных по своей сути решений. Эти решения требовали систематизации, обобщения на всевозможные ситуации, а также доступного описания, способствующего пониманию их людьми, которые до этого никогда их не использовали. Такое упорядочение знаний в объектно-ориентированном проектировании позволило бы повторно использовать готовые и уже проверенные решения, а не снова и снова «изобретать велосипед».

Решение проблемы систематизации накопленного опыта в объектно-ориентированном проектировании, а также представление его широкому кругу разработчиков и взяли на себя паттерны проектирования.

1.3. Понятие паттерна проектирования

В общем смысле паттерн представляет собой образец решения некоторой задачи так, что это решение можно использовать в различных ситуациях. В объектно-ориентированном проектировании паттерну можно дать следующие определение.

➤ *Под паттерном проектирования (design pattern) будем понимать описание взаимодействия объектов и классов, адаптированных для решения общей задачи проектирования в конкретном контексте [GoF95].*

Алгоритмы процедурного программирования также являются паттернами, но не проектирования, а вычислений, поскольку они решают не архитектурные, а вычислительные задачи.

Также следует подчеркнуть отличие паттернов проектирования от идиом. **Идиомы** — это паттерны, описыва-

ющие типичные решения на конкретном языке программирования, а паттерны проектирования не зависят от выбора языка (хотя их реализации, зачастую, зависимы от языка программирования).

В общем случае каждый паттерн состоит из таких составляющих:

1. **Имя** является уникальным идентификатором паттерна. Ссылка на него дает возможность описать задачу проектирования и подход к её решению. Имена паттернов проектирования, описанных в [GoF95], являются общепринятыми, поэтому, например, могут использоваться в проектной документации как ссылки на типичные архитектурные решения.
2. **Задача** описывает ситуацию, в которой можно применять паттерн. При формулировке задачи паттерна важно описать, в каком контексте она возникает.
3. **Решения** задачи проектирования в виде паттерна определяет общие функции каждого элемента дизайна и отношения между ними. При этом следует подчеркнуть, что рассматривается не конкретная, а общая ситуация, так как паттерн проектирования — это шаблон, который может применяться многократно для решения типичной задачи в различных контекстах.
4. **Результаты** представляют следствия применения паттерна. В них описываются преимущества и недостатки выбранного решения, его последствия, различного рода компромиссы, вариации паттерна. Также в результатах следует упомянуть об особенностях использования паттерна в контексте конкретного языка программирования.

Паттерн — это общее описание хорошего способа решения задачи. Рядом с понятием «паттерн» часто фигурирует понятие «антипаттерн», или «антишаблон». **Антипаттерн** — это часто повторяемое плохое решение, которое не рекомендуется использовать. Цель паттерна — распознать возможность применения хорошего решения проблемы. А цель антипаттерна — обнаружить плохую ситуацию и предложить подход к её устранению.

1.4. Практическое применение паттернов проектирования

Паттерны проектирования представляют общие решения типичных задач объектно-ориентированного проектирования. При этом может сложиться впечатление, что для формирования удачного дизайна системы в первую очередь необходимо спроектировать её части, основываясь на том или ином паттерне проектирования, иными словами, необходимо подвести дизайн системы под уже известные образцы. С практической точки зрения, отталкиваться от паттернов проектирования при разработке дизайна системы, не есть самым эффективным и гибким решением. Зачастую, такой подход может привести к правильному, с академической точки зрения, но не отвечающему требованиям гибкости и масштабируемости решению.

При разработке дизайна системы следует руководствоваться такими базовыми принципами:

- Всегда формировать простой дизайн: из двух предложенных решений, как правило, лучшим является то, что проще.

- Слабая зависимость: дизайн модуля должен быть таким, что бы в случае его модификации зависимые фрагменты системы не требовали или почти не требовали изменений.

«А где же тогда паттерны проектирования?» — спросите вы.

При таком подходе к проектированию логической структуры системы можно увидеть типичные задачи, которые уже решены с помощью паттернов проектирования. Тогда целесообразно применить уже готовое шаблонное решение, оценив при этом возможности и перспективы на основании его описания. Но здесь снова не следует забывать о принципах простого дизайна и слабой зависимости, так как излишнее желание воспользоваться паттерном проектирования может способствовать формированию плохого дизайна. Надо также помнить о том, что дизайн системы постоянно эволюционирует, поэтому успешно примененный паттерн со временем может трансформироваться в совсем другой или вовсе исчезнуть.

Нельзя не отметить, что благодаря паттернам проектирования произошла унификация терминологии: ссылки на них, как на комплексные решения задач дизайна, можно добавлять в проектную документацию, а также использовать в дискуссиях.

Практический опыт разработки программного обеспечения говорит, что любой полученный результат всегда подлежит тщательной проверке. Одним из индикаторов плохого дизайна системы могут быть следующие негативные явления в программном коде (*code smell*):

- **Дуближ кода:** одинаковый (или почти одинаковый код) в разных местах проекта.
- **Большие методы:** подпрограммы с большим количеством линий кода.
- **Большие классы:** на один класс положено слишком много, зачастую, разнородных функций.
- **Зависть:** класс чрезмерно использует методы другого класса.
- **Нарушение приватности:** класс чрезмерно зависит от деталей реализации другого класса.
- **Нарушение завещания:** класс переопределяет метод базового класса и при этом нарушает его контракт (т.е. первоначальное предназначение).
- **Ленивый класс:** класс, который делает слишком мало, т.е. класс с недостаточной или нецелостной функциональностью.
- **Чрезмерна сложность:** принудительное использование сложных решений, где простой дизайн был бы достаточным.
- **Чрезмерно длинные идентификаторы:** в том числе, использование длинных имен, для обозначения зависимостей, которые должны быть неявными.

При практическом применении паттернов проектирования также не следует забывать о том, что ООП постоянно развивается. Подходы, которые раньше считались образцовыми, со временем могут стать антипаттернами, а им на смену могут прийти значительно лучшие решения (как, например, случилось с паттерном [Singleton](#) (см. пункт 3.5)).

1.5. Классификация паттернов

В объектно-ориентированном анализе и проектировании (ООАП) на сегодняшний день разработано много различных паттернов, и паттерны проектирования (в частности, порождающие паттерны), рассматриваемые в этом уроке — это только одна из подкатегорий.

Для полноты картины вкратце рассмотрим классификацию всех паттернов ООАП:

1. **Архитектурные паттерны.** Описывают фундаментальные способы структурирования программных систем. Эти паттерны относятся к уровню систем и подсистем, а не классов. Паттерны этой категории систематизировал и описал К. Ларман.
2. **Паттерны проектирования.** Описывают структуру программных систем в терминах классов. Наиболее известными в этой области являются 23 паттерна, описанные в [GoF95].
3. **Паттерны анализа.** Представляют общие схемы организации процесса объектно-ориентированного моделирования.
4. **Паттерны тестирования.** Определяют общие схемы организации процесса тестирования программных систем.
5. **Паттерны реализации.** Описывают шаблоны, которые используются при написании программного кода.

Более детально рассмотрим паттерны проектирования, обобщив классификации, представленные в [Grand2004], [DPWiki], [DPOverview]:

1. **Основные паттерны** (*Fundamental Patterns*). Представляют наиболее важные фундаментальные паттерны, которые активно используются другими паттернами.
2. **Порождающие паттерны** (*Creational Patterns*). Определяют способы создания объектов в системе.
3. **Структурные паттерны** (*Structural Patterns*). Описывают способы построения сложных структур из классов и объектов.
4. **Поведенческие паттерны** (*Behavioral Patterns*). Описывают способы взаимодействия между объектами.
5. **Паттерны параллельного программирования** (*Concurrency Patterns*). Предназначены для координирования параллельных операций; решают такие проблемы как борьба за ресурсы и взаимоблокировка.
6. **Паттерны MVC**. Включает такие паттерны как MVC (*Model — View — Controller*), MVP (*Model — View — Presenter*) и др.
7. **Паттерны корпоративных систем** (*Enterprise Patterns*). Представляют решения для построения и интеграции больших корпоративных программных систем.

В работе [GoF95] представлено 23 паттерна проектирования. За своим назначением эти паттерны классифицируются так:

1. **Порождающие паттерны** (*Creational Patterns*). Абстрагируют процесс инстанцирования; делают систему независимой от того, как в ней создаются, komponуются и представляются объекты.

2. **Структурные паттерны** (*Structural Patterns*). Решают вопрос о создании из классов и объектов более крупных структур.
3. **Поведенческие паттерны** (*Behavioral Patterns*). Распределяют обязанности между объектами; описывают способы их взаимодействия.

В данном учебном курсе мы, главным образом, будем изучать паттерны из [GoF95], в частности, в текущем уроке детально рассмотрены порождающие паттерны.

2. Краткое введение в UML

Унифицированный язык визуального моделирования (*Unified Modeling Language* — UML) представляет собой средство для визуального представления элементов программной системы. Так как в процессе изучения паттернов проектирования рассматриваемые объектно-ориентированные модели удобно представлять графически в виде диаграмм классов UML, то нам важно рассмотреть их основные элементы.

2.1. Диаграмма классов

Диаграмма классов предназначена для представления статической структуры системы в терминах классов объектно-ориентированного программирования. В нашем учебном курсе все паттерны проектирования, а также примеры их использования описываются с использованием диаграмм этого типа.

Одним из основных составляющих диаграммы классов есть собственно класс. Графически он изображается в виде прямоугольника, разделенного на три части (рис. 2.1.1):

- 1) имя класса;
- 2) атрибуты (поля, члены данных) класса;
- 3) операции (методы, функции-члены) класса.

Секции 2 и 3 в изображении класса могут отсутствовать.

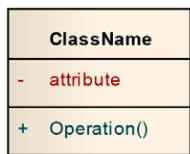


Рис. 2.1.1. Графическое изображение класса

В общем случае атрибут класса обозначается так
`[visibility] name [multiplicity] [: type] [= initial value]
 [{ property }],`

где

- **visibility** — область видимости, которая может получать следующие значения
 - ▷ `+-Public`
 - ▷ `#-Protected`
 - ▷ `--Private`
- **name** — имя атрибута (единственный обязательный параметр);
- **multiplicity** — кратность, т.е. количество экземпляров атрибута;
- **type** — тип, к которому принадлежит атрибут;
- **initial value** — начальное значение;
- **property** — другие свойства, например, `readonly`.

Полная форма определения операции класса имеет следующий вид:

`[visibility] name ([parameter list]) [: return type]
 [{ property }],`

где

- **visibility** — область видимости, которая может получать следующие значения

- ▷ +–Public
- ▷ #–Protected
- ▷ --Private
- **name** — имя операции;
- **parameter list** — список параметров, каждый элемент которого обозначается так:

[direct] name : type [= default value]

- ▷ **direct** — направление передачи значения:
- ▷ **in** — входящие
- ▷ **out** — выходящие
- ▷ **inout** — входящие и выходящие одновременно
- ▷ **name** — имя параметра;
- ▷ **type** — тип параметра;
- ▷ **default value** — значение по умолчанию;
- ▷ **return type** — тип возвращаемого значения;
- ▷ **property** — другие свойства операции.

Если класс или операция класса определены как абстрактные, то их название обозначается курсивом. Имена статических классов и статических членов класса обозначаются с подчеркиванием.

Интерфейсы на диаграмме классов отображаются двумя способами: полноформатно, как класс со стереотипом **Interface** (рис. 2.1.2), и в сокращенной круговой нотации (рис. 2.1.3).

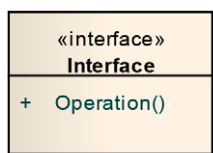


Рис. 2.1.2. Полноформатная нотация интерфейса



Рис. 2.1.3. Круговая нотация интерфейса

Наиболее важным для понимания аспектом в данном случае есть отношения между классами:

1. **Отношение наследования** (*Generalization Relationship*). Аналогично наследованию классов в объектно-ориентированном программировании. Обозначается линией со стрелкой в виде треугольника, стрелка направлена от подкласса к суперклассу (рис. 2.1.4).

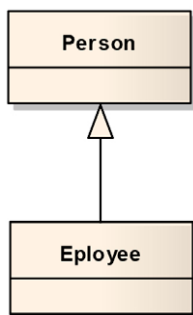


Рис. 2.1.4. Отношение наследования на диаграмме классов

2. **Отношение реализации** (*Realization Relationship*). Представляет собой частный случай отношения наследования. Используется для отображения реализации интерфейса классом. Графически изображается в виде пунктирной линии с треугольником на конце; треугольник указывает в сторону интерфейса (рис. 2.1.5).

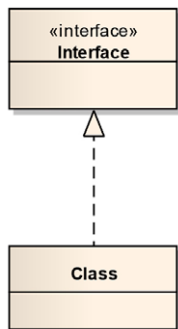


Рис. 2.1.5. Отношение реализации на диаграмме классов

3. Отношение зависимости (*Dependency Relationship*).

Обозначает факт использования заданным классом другого класса. Отношение зависимости может, например, применяться в таких случаях:

- ▶ класс содержит локальную переменную, принадлежащую к другому классу;
- ▶ класс использует объект другого класса в качестве параметра операции;
- ▶ метод класса возвращает объект другого класса;
- ▶ класс использует статическую операцию другого класса.

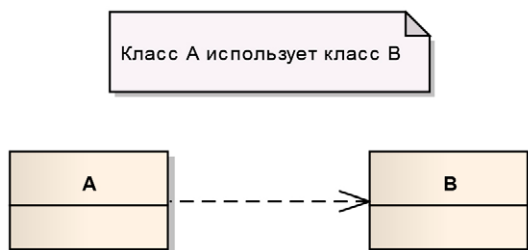


Рис. 2.1.6. Отношение зависимости на диаграмме классов

Графически отношение зависимости отображается в виде пунктирной стрелки, направленной от класса, который использует к классу, который используется (рис. 2.1.6).

4. **Отношение ассоциации** (*Association Relationship*). Обозначает некое структурное отношение между классами. Возникает между классами, например, когда, в одном классе в качестве поля содержится объект, принадлежащий к другому классу. Графически отображается в виде сплошной линии (рис. 2.1.7).

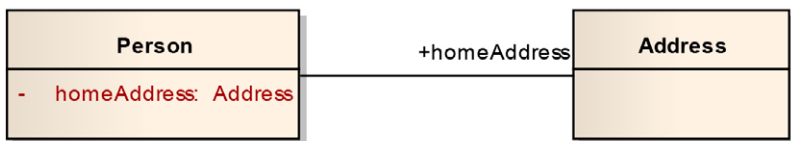


Рис. 2.1.7. Отношение ассоциации на диаграмме классов

5. **Отношение агрегации** (*Aggregation Relationship*). Представляет более строгую форму отношения ассоциации, служащую для обозначения логического включения при формировании целого из частей. Графически отображаться в виде сплошной линии с ромбом на конце со стороны класса, который обозначает целое (рис. 2.1.8).
6. **Отношение композиции** (*Composition Relationship*). Представляет более строгий случай отношения агрегации. Предусматривает включение части в целое, при этом часть не может существовать отдельно от целого. Графически отображается как агрегация только с той разницей, что ромб закрашивается в черный цвет (рис. 2.1.9).

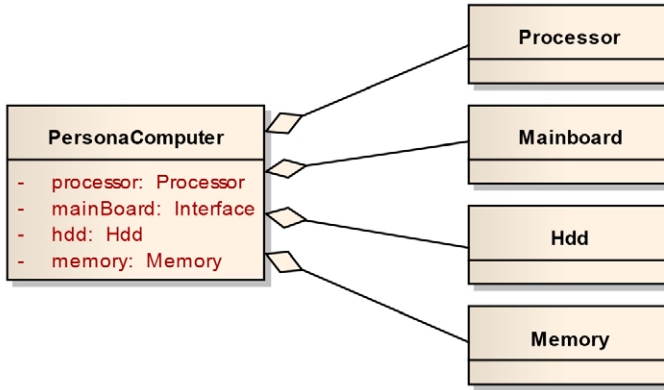


Рис. 2.1.8. Отношение агрегации на диаграмме классов

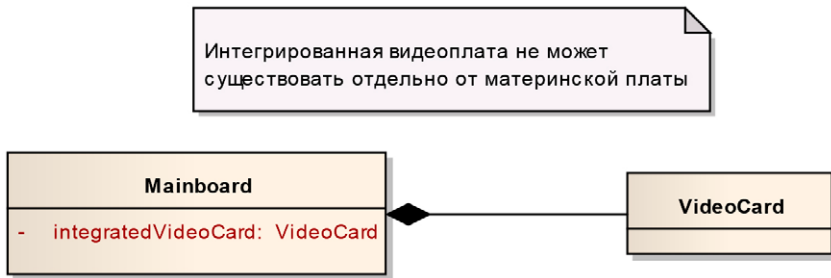


Рис. 2.1.9. Отношение композиции на диаграмме классов

2.2. Диаграмма объектов

В отличие от диаграммы классов, которая отображает статическую структурную модель системы, диаграмма объектов представляет экземпляры классов и отношения между ними в некоторый момент работы программы. Иными словами, диаграмма объектов — это снимок состояния работающей системы в заданный момент времени.

В общем случае имя объекта на диаграмме отображается с подчеркиванием и имеет вид:

Имя объекта: Имя класса

При этом одна из составляющих имени может не указываться.

Простейший пример диаграммы объектов изображен на рис. 2.2.10.

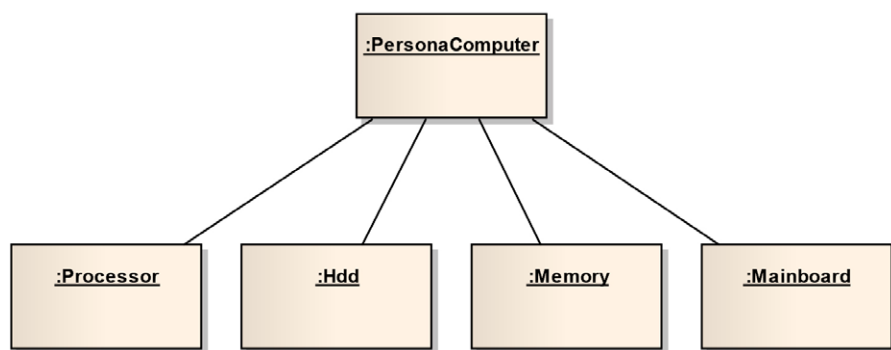


Рис. 2.2.10. Пример диаграммы объектов

Представленные выше диаграммы классов и объектов будут использоваться нами в процессе изучения всех паттернов проектирования. Все примеры этого курса написаны на Java. К уроку приложена папка с примерами реализации паттернов.

3. Порождающие паттерны

Порождающие паттерны проектирования абстрагируют процесс инстанцирования. Они помогают сделать систему независимой от способа создания, композиции, и представления объектов. Паттерны этой категории позволяют ответить на вопрос: кто, когда и как создает объекты в системе [GoF95].

3.1. Abstract Factory

3.1.1. Название паттерна

Abstract Factory — Абстрактная фабрика. Также известен под именем: **Toolkit** — Инструментарий.

Описан в работе [GoF95].

3.1.2. Цель паттерна

Предоставляет интерфейс для создания семейства взаимосвязанных и взаимозависимых объектов, не специфицируя конкретных классов, объекты которых будут создаваться. [GoF95]

3.1.3. Паттерн следует использовать, когда...

- Система не должна зависеть от того, как в ней создаются и komponуются объекты.
- Объекты, входящие в семейство, должны использоваться вместе.
- Система должна конфигурироваться одним из семейств объектов.
- Надо предоставить интерфейс библиотеки, не раскрывая её внутренней реализации.

3.1.4. Причины возникновения паттерна

Перед нами стоит задача разработать игру «Супер-Ралли», суть которой, конечно же, заключается в гонках на автомобилях. Одно из основных функциональных требований гласит, что игрок должен иметь возможность выбирать себе автомобиль для участия в гонках. Каждый из предложенных игровых автомобилей состоит из специфического набора составляющих (двигателя, колес, кузова, коробки передач, бензобака), которые в совокупности определяют возможности автомобиля (скорость, маневренность, устойчивость к повреждениям, длительность непрерывной гонки без перезаправки и д.р.). Так как разных типов автомобилей в игре может быть много, более того, их количество может изменяться динамически (например, в зависимости от опыта игрока), то, с технической точки зрения, клиентский код, реализующий конфигурирование автомобиля специфичным семейством составляющих, не должен зависеть от типа выбранного автомобиля.

Для реализации этого требования предлагается следующие. Рассмотрим интерфейс «ФабрикаСоставляющих», предназначение которого — представить интерфейс для конкретных классов-фабрик, которые будут создавать семейства составляющих для каждого конкретного типа автомобиля. Методы этого класса должны возвращать ссылки на абстрактные составляющие, что позволит в конкретных классах-фабриках, создавать конкретные составляющие (подклассы абстрактных составляющих). Диаграмма классов, демонстрирующая этот поход, изображена на рис. 3.1.1.

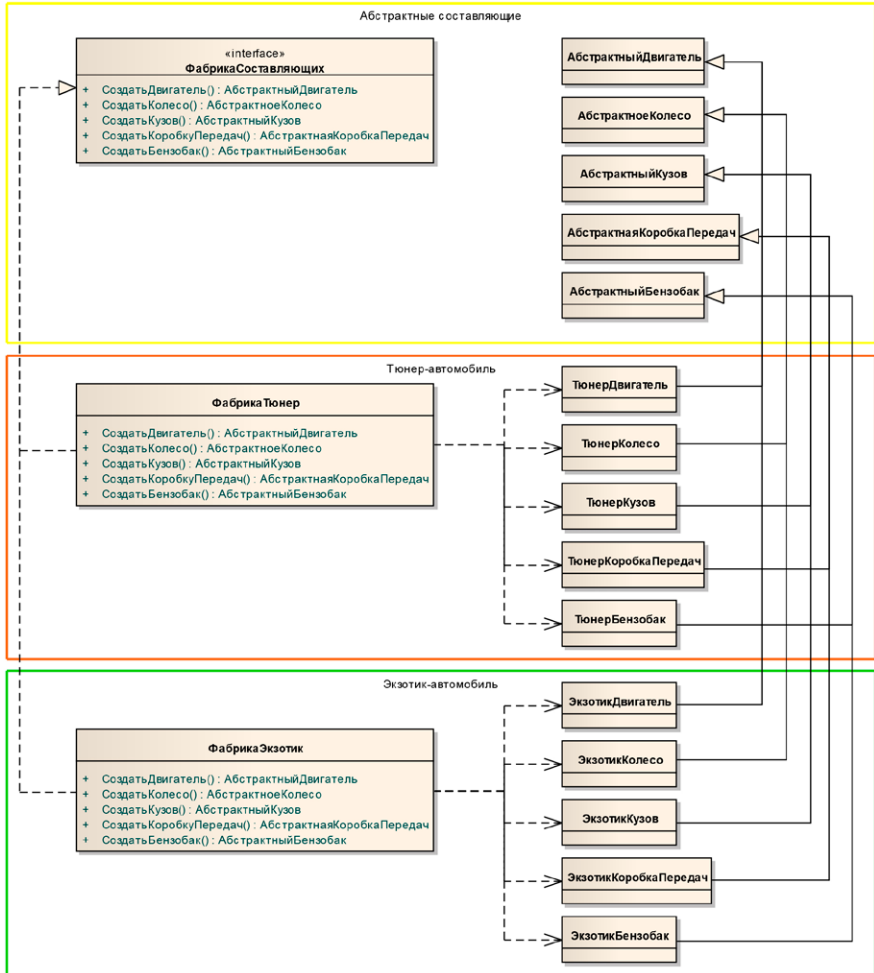


Рис. 3.1.1. Фабрики составляющих игровых автомобилей

Клиентский код, который «собирает» автомобиль с деталей, конфигурируется интерфейсной ссылкой «ФабрикаСоставляющих», методы которой возвращают ссылки на абстрактные составляющие. Таким образом, мы имеем возможность передавать клиенту объект кон-

кретной фабрики, которая создает семейство объектов конкретных составляющих.

Существенный недостаток продемонстрированного подхода: определение интерфейса «Фабрика Составляющих» на этапе компиляции предусматривает создание фиксированного набора продуктов, из чего следует:

- 1) каждая из конкретных фабрик должна создавать конкретные продукты, соответствующие этому набору, т.е. количество и тип продуктов жестко определяется на этапе компиляции и не может меняться на этапе выполнения;
- 2) при необходимости добавления нового типа продуктов нужно изменять абстрактную фабрику и каждую из конкретных фабрик.

Во избежание таких проблем в качестве альтернативы абстрактной фабрике можно рассматривать паттерн прототип (см. пункт 3.4).

Также следует отметить, что при реализации данного подхода каждую из конкретных фабрик можно определить как **Singleton** (см. пункт 3.5), так как в программе, скорее всего, не будет надобности создавать более одного экземпляра конкретной фабрики.

3.1.5. Структура паттерна

Общая структура паттерна **Abstract Factory** представлена на рис. 3.1.2.

Участники паттерна:

- **AbstractFactory** — абстрактная фабрика
 - ▷ Представляет общий интерфейс для создания семейства продуктов.

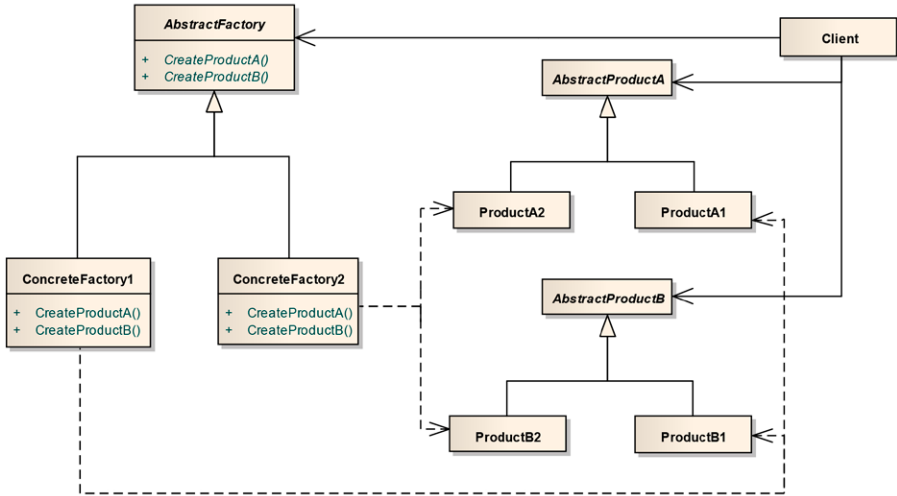


Рис. 3.1.2. Общая структура паттерна Abstract Factory

- **ConcreteFactory** — конкретная фабрика
 - ▷ Реализует интерфейс **AbstractFactory** и создает семейство конкретных продуктов.
- **AbstractProduct** — абстрактный продукт
 - ▷ Представляет интерфейс абстрактного продукта, ссылку на который возвращают методы фабрик.
- **ConcreteProduct** — конкретный продукт
 - ▷ Реализует конкретный тип продукта, который создается конкретной фабрикой.

Отношения между участниками:

- Клиент знает только о существовании абстрактной фабрики и абстрактных продуктов.
- Для создания семейства конкретных продуктов клиент конфигурируется соответствующим экземпляром конкретной фабрики.

- Методы конкретной фабрики создают экземпляры конкретных продуктов, возвращая их в виде ссылок на соответствующие абстрактные продукты.

3.1.6. Результаты использования паттерна

- **Позволяет изолировать конкретные классы продуктов.** Клиент знает о существовании только абстрактных продуктов, что ведет к упрощению его архитектуры.
- **Упрощает замену семейств продуктов.** Для использования другого семейства продуктов достаточно конфигурировать клиентский код, соответствующий конкретной фабрикой.
- **Дает гарантию сочетаемости продуктов.** Так как каждая конкретная фабрика создает группу продуктов, то она и следит за обеспечением их сочетаемости.
- **Серьезным недостатком паттерна есть трудность поддержки нового вида продуктов.** Для добавления нового продукта необходимо изменять всю иерархию фабрик, а также клиентский код.

3.1.7. Практический пример использования паттерна

Пусть стоит задача разработать программное обеспечение для магазина компьютерной техники. По мнению заказчика, одной из наиболее востребованных возможностей программы будет возможность быстрого создания конфигурации системного блока.

Для упрощения изложения предположим, что в состав конфигурации системного блока входят:

- 1) бокс (*Box*);
- 2) процессор (*Processor*);

- 3) системная плата (*MainBoard*);
- 4) жесткий диск (*Hdd*);
- 5) оперативная память (*Memory*).

Для каждой из этих составляющих определим абстрактный класс. Конкретные модели составляющих будем определять путем наследования от абстрактного базового класса. (Сразу же следует отметить, что такой подход не самый лучший с практической точки зрения).

Класс, представляющий конфигурацию системного блока, назовем PC (листинг 3.1.1).

Листинг 3.1.1. Класс PC

```
// класс, описывающий компьютер и его составляющие
public class PC {

    Box box;
    Processor processor;
    MainBoard mainBoard;
    Hdd hdd;
    Memory memory;

    public PC() {
        box = null;
        processor = null;
        mainBoard = null;
        hdd = null;
        memory = null;
    }

    public Box GetBox() {
        return box;
    }

    public void SetBox(Box pBox) {
        box = pBox;
    }
}
```

```
public Processor GetProcessor() {  
    return processor;  
}  
public void SetProcessor(Processor pProcessor) {  
    processor = pProcessor;  
}  
public MainBoard GetMainBoard() {  
    return mainBoard;  
}  
public void SetMainBoard(MainBoard pMainBoard) {  
    mainBoard = pMainBoard;  
}  
public Hdd GetHdd() {  
    return hdd;  
}  
public void SetHdd(Hdd pHdd) {  
    hdd = pHdd;  
}  
public Memory GetMemory() {  
    return memory;  
}  
public void SetMemory(Memory pMemory) {  
    memory = pMemory;  
}  
}
```

Допустим, что наша программа должна создавать шаблоны типичных конфигураций двух типов: домашняя и офисная.

С логической точки зрения, важно, чтобы все составляющие заданной конфигурации системного блока работали согласованно, поэтому ответственность за их создание надо положить на один класс-фабрику. Эта фабрика должна реализовать интерфейс *IPCFactory*

(листинг 3.1.2). Заметим, что методы этого интерфейса возвращают ссылки на классы абстрактных продуктов.

Листинг 3.1.2. Интерфейс IPCFactory

```
/*
 * Интерфейс фабрики для создания конфигурации
 * системного блока персонального компьютера
 */

interface IPCFactory {
    Box CreateBox();
    Processor CreateProcessor();
    MainBoard CreateMainBoard();
    Hdd CreateHdd();
    Memory CreateMemory();
}
```

Для создания наборов компонентов типичных конфигураций определим классы конкретных фабрик **HomePcFactory** (листинг 3.1.3) и **OfficePcFactory** (листинг 3.1.4). В каждом из create-методов этих классов создается объект конкретного класса продукта, соответствующего типу конфигурации.

Листинг 3.1.3. Класс HomePcFactory

```
/*
 * Фабрика для создания "домашней" конфигурации
 * системного блока персонального компьютера
 */

class HomePcFactory implements IPCFactory {
    @Override
    public Box CreateBox(){
        return new SilverBox();
    }
}
```

```

@Override
public Processor CreateProcessor() {
    return new IntelProcessor();
}
@Override
public MainBoard CreateMainBoard() {
    return new MSIMainBord();
}
@Override
public Hdd CreateHdd() {
    return new SamsungHDD();
}
@Override
public Memory CreateMemory() {
    return new Ddr2Memory();
}
}

```

Листинг 3.1.4. Класс OfficePcFactory

```

/*
 * Фабрика для создания "офисной" конфигурации
 * системного блока персонального компьютера
 */
class OfficePcFactory implements IPCFactory {
    @Override
    public Box CreateBox() {
        return new BlackBox();
    }
    @Override
    public Processor CreateProcessor() {
        return new AmdProcessor();
    }
    @Override
    public MainBoard CreateMainBoard() {
        return new AsusMainBord();
    }
}

```

```
@Override
public Hdd CreateHdd() {
    return new LGHDD();
}

@Override
public Memory CreateMemory() {
    return new DdrMemory();
}
}
```

Определим класс **PCConfigurator**, отвечающий за конфигурирование объекта типа РС выбранным семейством составляющих (листинг 3.1.5).

Листинг 3.1.5. Класс PCConfigurator

```
// класс конфигуратор
public class PCConfigurator
{
    /*
     * Фабрика составляющих персонального компьютера
     */

    private IPCFactory pcFactory;

    public PCConfigurator() {
        pcFactory = null;
    }
    public IPCFactory GetPCFactory() {
        return pcFactory;
    }
    public void SetPCFactory(IPCFactory
                             pcCurrentFactory) {
        pcFactory = pcCurrentFactory;
    }
}
```

```
/*
 * Метод конфигурирования системного блока
 */
public void Configure(PC pc)
{
    pc.SetBox(pcFactory.CreateBox());
    pc.SetMainBoard(pcFactory.CreateMainBoard());
    pc.SetHdd(pcFactory.CreateHdd());
    pc.SetMemory(pcFactory.CreateMemory());
    pc.SetProcessor(pcFactory.CreateProcessor());
}
}
```

Класс **PCConfigurator** принимает экземпляр конкретной фабрики и с помощью её методов создает составляющие персонального компьютера. При этом следует подчеркнуть, что **PCConfigurator** работает с интерфейсной ссылкой **IPCFactory**, т.е. он ничего не знает о конкретных фабриках конфигураций и конкретных составляющих. В этом и проявляется вся сила абстрактной фабрики — конкретную фабрику можно определять на этапе выполнения программы, и при этом клиентский код (в данном случае **PCConfigurator**) не зависит от конкретных фабрик или конкретных продуктов.

Полная диаграмма классов представленной реализации представлена на рис. 3.1.3.

В представленной реализации легко увидеть типичный недостаток абстрактной фабрики: при изменении количества продуктов необходимо модифицировать всю иерархию фабрик, а также клиентский код.

Еще один подход (возможно, более гибкий) к реализации задачи типичных конфигураций персонального

компьютера представлен в п. 3.4. Он базируется на использовании паттерна *Prototype*.

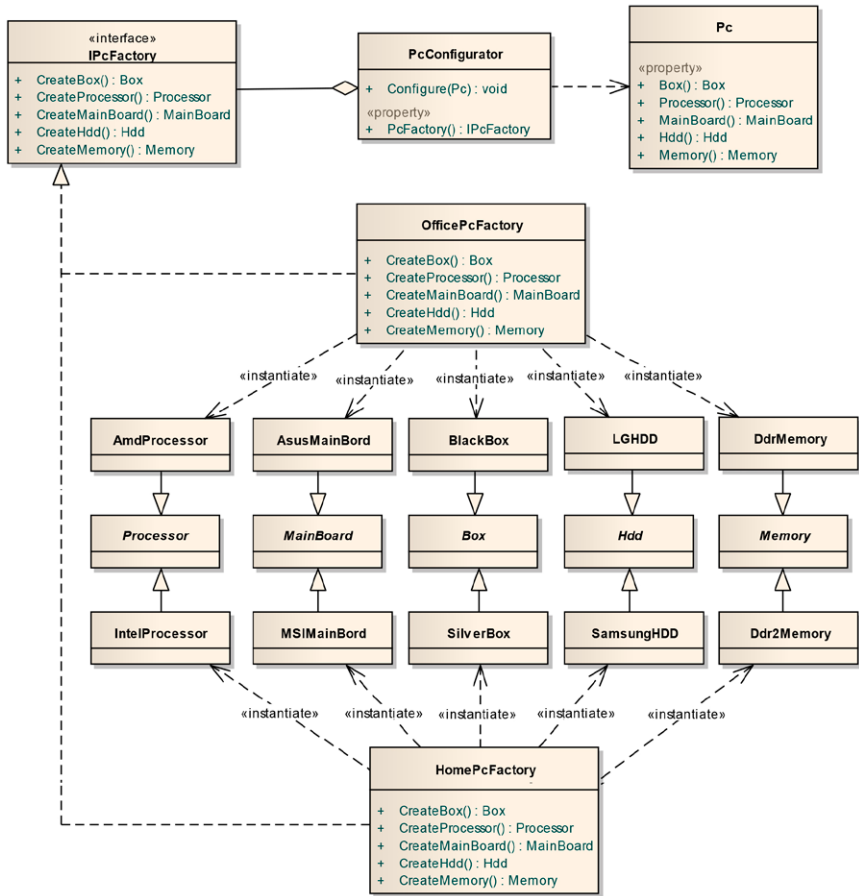


Рис. 3.1.3. Модель классов конфигуратора системного блока персонального компьютера

К уроку приложена папка с примерами реализации паттернов. Порождающие паттерны расположены в папке *Creational*. Полный код примера находится в папке *AbstractFactory*.

3.2. Builder

3.2.1. Название паттерна

Builder/Строитель.

Описан в работе [GoF95].

3.2.2. Цель паттерна

Отделяет процесс конструирования сложного объекта от его представления так, что в результате одного и того же процесса конструирования получаются разные представления [GoF95]. Иными словами, клиентский код может создавать сложный объект, определяя для него не только тип, но и содержимое. При этом клиент не должен знать о деталях конструирования объекта [Grand2004].

3.2.3. Паттерн следует использовать, когда...

- Общий алгоритм построения сложного объекта не должен зависеть от специфики каждого из его шагов.
- В результате одного и того же алгоритма конструирования надо получить различные продукты.

3.2.4. Причины возникновения паттерна

Представим себе, что мы имеем конвейер для выпуска автомобилей. Смысл конвейера заключается в пошаговом построении сложного продукта, которым в данном случае является автомобиль. Конвейер определяет общую последовательность шагов (т.е. алгоритм) конструирования. При этом специфика каждого из шагов определяется, главным образом, моделью собираемого автомобиля. Такое разделение общего алгоритма построения и специфики каждого из шагов позволят компании значительно сэкономить: на

одном и том же конвейере могут выпускаться автомобили разных моделей с различными характеристиками.

С общей технологической точки зрения, автомобиль на конвейере проходит такие этапы:

- 1) сборка кузова;
- 2) установка двигателя;
- 3) установка колес;
- 4) покраска;
- 5) подготовка салона.

Технические детали процессов, происходящих на каждом шаге, известны уже конкретной технологии производства модели автомобиля. Пусть завод может производить автомобили следующих моделей: автомобили класса «мини», спортивные автомобили, вседорожники. При такой постановке производственного процесса для компании не составит проблем дополнить спектр выпускаемых моделей новыми образцами без изменений общего конвейерного цикла.

Перенесемся теперь в мир объектно-ориентированного программирования.

Определим класс «Конвейер», который будет прототипом реального конвейера, т.е. будет определять общую последовательность шагов конструирования. Метод «Собрать» этого класса будет исполнять процесс конструирования посредством выполнения этих шагов без зависимости от технических деталей реализации каждого шага.

Ответственность за реализацию шагов конструирования положим на абстрактный класс, который назовем «ТехнологияМодели». В результате применения конкретных подклассов класса «ТехнологияМодели» мы получаем на

выходе разные модели автомобилей, т.е. экземпляры разных классов автомобилей. В нашем случае определим такие подклассы класса «ТехнологияМодели»: «ТехнологияМиниАвто», «ТехнологияСпортивныйАвто», «ТехнологияВседорожныйАвто». Каждая из этих технологий соответственно предусматривает выпуск таких моделей автомобилей: «МиниАвто», «СпортивныйАвто», «ВседорожныйАвто».

Для начала производства автомобиля необходимо задать конкретную технологию для конвейера и вызвать метод «Собрать». После завершения процесса сборки готовый автомобиль можно получить у объекта технологии с помощью метода «ПолучитьРезультат()».

Диаграмма классов описанной объектно-ориентированной модели представлена на рис. 3.2.1.

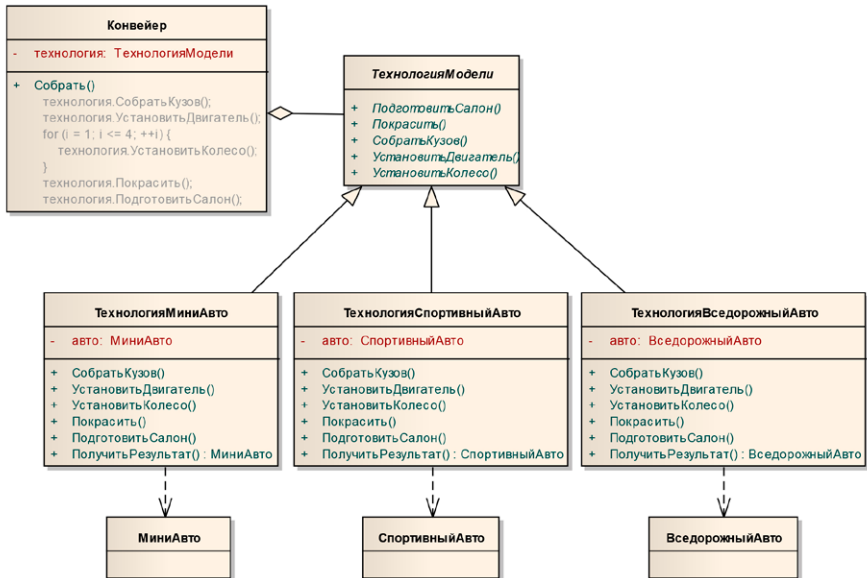


Рис. 3.2.4. Диаграмма классов модели конвейера по производству автомобилей

Построенная модель обладает рядом преимуществ:

1. Конкретная технология конструирования строится по общему шаблону, реализуя действия, которые он определяет;
2. Общий алгоритм процесса конструирования не зависит от деталей, специфических для конкретной технологии;
3. Есть возможность без опасности роста сложности структуры модели реализовать под общий алгоритм большое количество конкретных технологий.

3.2.5. Структура паттерна

В общем случае структура паттерна **Builder** имеет вид, представленный на рис. 3.2.2.

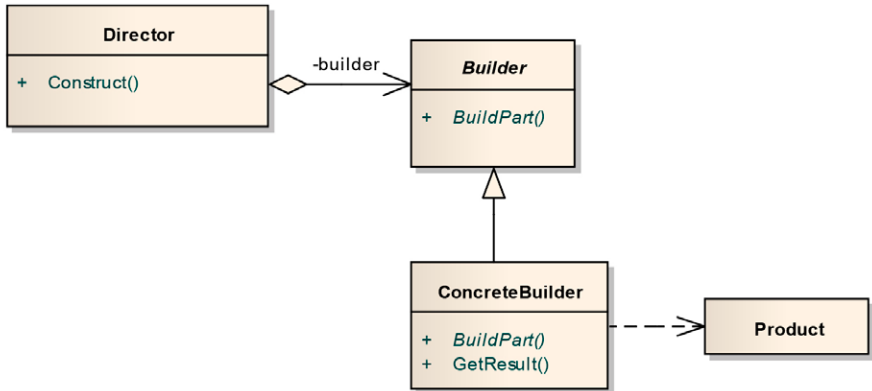


Рис. 3.2.5. Структура паттерна Builder

Участники паттерна:

- **Builder** (*Технология Модели*) — строитель
 - ▷ Обеспечивает интерфейс для пошагового конструирования сложного объекта (продукта) из частей.

- **ConcreteBuilder** (*ТехнологияМиниАвто* и др.) — конкретный строитель
 - ▷ Реализует шаги построения сложного объекта, определенные в базовом классе **Builder**.
 - ▷ Создает результат построения (Product) и следит за пошаговым конструированием.
 - ▷ Определяет интерфейс для доступа к результату конструирования
- **Director** (*Конвейер*) — распорядитель
 - ▷ Определяет общий алгоритм конструирования, используя для реализации отдельных шагов возможности класса **Builder**.
- **Product** (*МиниАвто* и др.) — продукт
 - ▷ Сложный объект, который получается в результате конструирования.

Отношения между участниками:

- Клиент конфигурирует распорядителя (**Director**) экземпляром конкретного строителя.
- Распорядитель вызывает методы строителя для конструирования частей продукта.
- Конкретный строитель создает продукт и следит за его конструированием.
- Конкретный строитель представляет интерфейс для доступа к продукту.

3.2.6. Результаты использования паттерна

Есть возможность изменять внутреннюю структуру создаваемого продукта (или создать новый продукт). Так как продукт конструируется через абстрактный ин-

терфейс класса **Builder**, для добавления нового продукта достаточно определить новый вид строителя (т.е. реализовать новый подкласс класса **Builder**).

Повышение модульности за счет разделения распорядителя и строителя. Каждый строитель имеет весь необходимый код для пошагового построения продукта. Поэтому он может использоваться разными распорядителями для построения вариантов продукта из одних и тех же частей.

Пошаговое построение продукта позволяет обеспечить более пристальный контроль над процессом конструирования (в отличие от других порождающих паттернов, которые создают продукт мгновенно).

3.2.7. Практический пример использования паттерна

Предположим нам надо разработать код для создания различных летальных устройств. Они, как вы понимаете бывают разные 😊. Представим, что у нас есть два вида летальных устройств: дельтаплан и планер. Безусловно нам нужно будет создать обобщенный класс летального устройства. Назовем его **Aircraft**. Его код приводим ниже:

Листинг 3.2.1. Класс Aircraft

```
// базовый класс для летального устройства
class Aircraft
{
    public Aircraft(String type) {
        aircraftType = type;
        parts = new HashMap<String, String>();
    }
    // тип летательного устройства
    private String aircraftType;
```

```
// хранилище информации об устройстве
HashMap<String, String> parts;

// получение информации о конкретной части устройства
public String GetPart(String key) throws Exception {
    if (!CheckForPart(key)) {
        throw new Exception("There is no such key!");
    }
    return parts.get(key);
}
// установка значения для конкретной части
// устройства

public void SetPart(String key, String value) {
    parts.put(key, value);
}
// проверка на наличие части

public boolean CheckForPart(String key) {
    return parts.get(key) != null? true:false;
}
// отображение информации об летательном устройстве

public void Show() {
    System.out.println("\n=====");

    System.out.println("Aircraft Type:" +
        aircraftType+"\n");

    System.out.println("Frame:" + parts.get("frame") +
        "\n");

    System.out.println("Engine:" +
        parts.get("engine") + "\n");

    System.out.println("Wheels:" +
        parts.get("wheels") + "\n");
}
```

```

        System.out.println("Doors:" +
                           parts.get("doors") + "\n");
    }
}

```

Теперь перейдем к классам конкретных строителей. Эти классы будут отвечать за создание двух типов самолетов: дельтаплана и планера. Начнем с абстрактного класса строителя. Он описывает конкретные операции для классов-строителей:

Листинг 3.2.1. Класс строителя базовый

```

abstract class AircraftBuilder
{
    protected Aircraft aircraft;

    public AircraftBuilder() {

    }

    public Aircraft GetAircraft() {
        return aircraft;
    }

    public abstract void BuildFrame();
    public abstract void BuildEngine();
    public abstract void BuildWheels();
    public abstract void BuildDoors();
    public abstract void BuildWings();
}

```

Теперь опишем класс конкретного строителя. Это будет строитель, который умеет строить дельтапланы.

Листинг 3.2.2. Класс строителя дельтапланов

```
class HangGliderBuilder extends AircraftBuilder
{
    public HangGliderBuilder() {
        aircraft = new Aircraft("Hang Glider");
    }
    public void BuildFrame() {
        aircraft.SetPart("frame", "Hang glider frame");
    }
    public void BuildEngine() {
        aircraft.SetPart("engine", "no engine");
    }
    public void BuildWheels() {
        aircraft.SetPart("wheels", "no wheels");
    }
    public void BuildDoors() {
        aircraft.SetPart("doors", "no doors");
    }
    public void BuildWings() {
        aircraft.SetPart("wings", "1");
    }
}
```

Теперь реализуем код строителя, который умеет создавать планеры.

Листинг 3.2.3. Класс строителя планеров

```
// Класс строителя. Умеет создавать планеры
class GliderBuilder extends AircraftBuilder
{
    public GliderBuilder() {
        aircraft = new Aircraft("Glider");
    }
}
```

```

public void BuildFrame() {
    aircraft.SetPart("frame", "Glider frame");
}
public void BuildEngine() {
    aircraft.SetPart("engine", "no engine");
}
public void BuildWheels() {
    aircraft.SetPart("wheels", "1");
}
public void BuildDoors() {
    aircraft.SetPart("doors", "1");
}
public void BuildWings() {
    aircraft.SetPart("wings", "2");
}
}

```

А теперь создадим класс директора, который умеет использовать конкретного строителя для построения летательного аппарата.

Листинг 3.2.4. Класс директора

```

class AircraftConstructor
{
    public AircraftConstructor() {

    }
    public void Construct(AircraftBuilder
                        aircraftBuilder) {
        aircraftBuilder.BuildFrame();
        aircraftBuilder.BuildEngine();
        aircraftBuilder.BuildWheels();
        aircraftBuilder.BuildDoors();
    }
}

```

А теперь давайте рассмотрим код внутри `main` для понимания того, как будет работать наш пример.

Листинг 3.2.5. Тело функции `main`

```
public class Run {  
    public static void main(String[] args) {  
        try {  
            // Создаем объект класса директора  
            AircraftConstructor shop =  
                new AircraftConstructor();  
  
            // Создаем объект класса строителя.  
            // Этот объект умеет создавать дельтапланы  
            AircraftBuilder builder =  
                new HangGliderBuilder();  
  
            // сооружаем дельтаплан  
            shop.Construct(builder);  
  
            // показываем информацию о дельтаплане  
            builder.GetAircraft().Show();  
  
            // Создаем объект класса строителя.  
            // Этот объект умеет создавать планеры  
            builder = new GliderBuilder();  
  
            // сооружаем планер  
            shop.Construct(builder);  
  
            // показываем информацию о планере  
            builder.GetAircraft().Show();  
        }  
        catch(Exception ex) {  
            System.out.println(  
                "Exception happened!\nException  
description:\n" +  
                ex.getMessage());  
        }  
    }  
}
```


*К уроку приложена папка с примерами реализации паттернов. Порождающие паттерны расположены в папке *Creational*. Полный код примера смотрите в папке под названием *Builder*.*

3.3. Factory Method

3.3.1. Название паттерна

Factory Method — Фабричный метод.

Также известный под именем: **Virtual Constructor** — Виртуальный конструктор.

Описан в работе [GoF95].

3.3.2. Цель паттерна

Определяет интерфейс для создания объекта, но оставляет за своими подклассами решение, какой объект надо создавать [GoF95].

3.3.3. Паттерн следует использовать, когда...

- Класс не должен зависеть от конкретного типа создаваемого продукта.
- Классу не известен конкретный тип продукта, который ему надо создавать.
- Конкретные типы создаваемых продуктов могут/должны определяться в подклассах.

3.3.4. Причины возникновения паттерна

Пусть разрабатываться игра типа «стрелялка». В процессе игры герой может выбрать разные типы огнестрельного оружия. Тип оружия определяет частоту выстрелов, калибр пули, тип пули (например, обычная

или разрывная). Для простоты изложения идеи будем считать, что конкретный тип оружия может стрелять только одним типом пули.

Стоит следующий вопрос. Как связать конкретный класс оружия с типом пули, которыми он стреляет?

Логично было бы положить обязанность за создание экземпляра пули на объект класса оружия, из которого производится выстрел. Таким образом, каждый из конкретных типов оружия будет знать, к какому классу принадлежат его пули. Диаграмма классов, демонстрирующая такой подход, представлена на рис. 3.3.2.

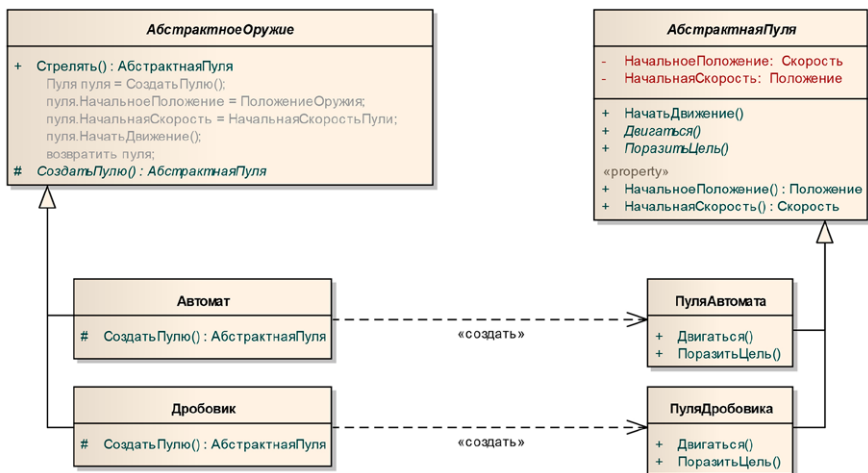


Рис. 3.3.6. Реализация взаимосвязи между типами оружия и типами пуль

Изложенный подход обладает следующими преимуществами:

1. Клиентскому коду не надо заботиться о конфигурировании экземпляра оружия специфическим ему типом пули;

2. Каждая пуля при выстреле конфигурируется параметрами (начальная скорость, начальное положение), о которых знает только оружие, из которого производится выстрел;
3. Клиентскому коду должно быть известно только об абстрактной пуле, т.е. с каждым конкретным экземпляром пули клиент взаимодействует через интерфейс АбстрактнаяПуля (что позволяет упростить его архитектуру).

Один из основных недостатков предложенного подхода заключается в том, что тип пули для конкретного класса оружия определяется статично на этапе компиляции и не может изменяться на этапе выполнения.

Реализация продемонстрированной идеи изложена в п. 3.3.5.

3.3.5. Структура паттерна

Общая структура паттерна **Factory Method**, представлена на рисунке 3.3.2.

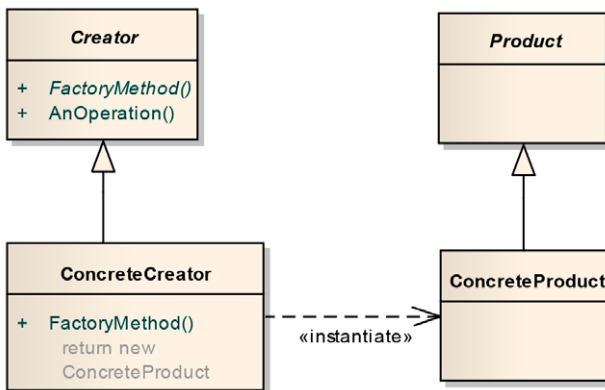


Рис. 3.3.7. Общая структура паттерна Factory Method

Участники паттерна:

- **Creator** (*АбстрактноеОружие*) — абстрактный создатель
 - ▷ Представляет абстрактный метод для создания экземпляра продукта, т.е. делегирует создание продукта своим подклассам.
- **ConcreteCreator** (*Автомат, Дробовик*) — конкретный создатель
 - ▷ Реализует метод создания экземпляра продукта.
- **Product** (*АбстрактнаяПуля*) — абстрактный продукт
 - ▷ Представляет абстрактный интерфейс продукта, через который с ним работает Creator.
- **ConcreteProduct** (*ПуляАвтомата, ПуляДробовика*) — конкретный продукт
 - ▷ Реализует интерфейс абстрактного продукта.

Отношения между участниками:

- **Creator** представляет абстрактный метод **FactoryMethod()** для создания экземпляра продукта, который возвращает ссылку на **Product**.
- **Creator** возлагает ответственность за создание экземпляра конкретного продукта на свои подклассы.
- **ConcreteCreator** реализует метод **FactoryMethod()**, обеспечивая создание объекта класса **ConcreteProduct**.

3.3.6. Результаты использования паттерна

Класс **Creator** не зависит от конкретного типа создаваемых продуктов.

За создание продукта отвечают подклассы. В этом аспекте есть как преимущества, так и недостатки. Пре-

имущества заключаются в том, что класс `Creator` постепенно уточняет конкретные продукты в своих подклассах. **Недостаток:** для реализации возможности создавать новый тип продуктов необходимо создавать новый подкласс `Creator`, что может привести к неоправданному росту числа его подклассов.

Позволяет объединить две параллельные иерархии классов создателей и продуктов.

Возможно определение реализации фабричного метода по умолчанию. Зачастую, фабричный метод представляется как абстрактный, что требует его обязательного определения в подклассах. Если для него определить реализацию по умолчанию, то он будет дополнительной возможностью модифицировать возможности класса при наследовании.

Конкретный класс создаваемого продукта может передаваться как тип-параметр класса `Creator` или его подклассов. В таком случае фабричный метод будет создавать экземпляры обобщенного типа-параметра, и для определения нового типа продуктов не обязательно прибегать к наследованию. Такой вариант реализации может быть легко использован в C++ или C#, но будет неприемлем, например, для Java (по крайней мере, для версий 1.6 и младше).

3.3.7. Практический пример использования паттерна

Реализуем идею, представленную в п. 3.3.2.

Описание классов пуль представлено в листинге 3.3.1. Следует обратить внимание на абстрактные методы `Hit-Target()`, который реализует попадание в цель, и `Move-`

ment()), реализующий траекторию полета пули. Реализация этих методов определяется в подклассах класса **AbstractBullet**. Это значит, что наследование от класса **AbstractBullet**, в частности, обеспечивает реализацию поведения, специфического для конкретного типа пули.

Листинг 3.3.1. Реализация классов пуль

```
/*
 * Точка в трехмерном пространстве.
 * Используется для определения положения.
 */
class Point3D {
    private int X;
    private int Y;
    private int Z;

    public Point3D(int px, int py, int pz) {
        X = px;
        Y = py;
        Z = pz;
    }
    public int getX() {
        return X;
    }
    public void setX(int px) {
        X = px;
    }
    public int getY() {
        return Y;
    }
    public void setY(int py) {
        Y = py;
    }
    public int getZ() {
        return Z;
    }
}
```

```

        public void setZ(int pz) {
            Z = pz;
        }
    }

    /*
     * Вектор в трехмерном пространстве.
     * Используется для определения направления.
     */
    class Vector3D {
        private int X;
        private int Y;
        private int Z;

        public Vector3D(int px, int py, int pz) {
            X = px;
            Y = py;
            Z = pz;
        }
        public int getX() {
            return X;
        }
        public void setX(int px) {
            X = px;
        }
        public int getY() {
            return Y;
        }
        public void setY(int py) {
            Y = py;
        }
        public int getZ() {
            return Z;
        }
        public void setZ(int pz) {
            Z = pz;
        }
    }
}

```

```
/*
 * Класс абстрактной пули.
 */
abstract class AbstractBullet
{
    private Point3D location;
    private Vector3D direction;
    private double caliber;

    /*
     * Текущее положение пули
     */
    public Point3D GetLocation() {
        return location;
    }
    public void SetLocation(Point3D newLocation) {
        location = newLocation;
    }

    /*
     * Направление пули
     */
    public Vector3D GetDirection() {
        return direction;
    }
    public void SetDirection(Vector3D newDirection) {
        direction = newDirection;
    }

    /*
     * Калибр пули
     */
    public double GetCaliber() {
        return caliber;
    }
    public void SetCaliber(double newCaliber) {
        caliber = newCaliber;
    }
}
```



```

/*
 * Начало движения пули.
 */
public void StartMovement()
{
    // Реализация начала движения
}

/*
 * Метод поражения цели.
 * Так как разные типы пуль поражают цель по-разному,
 * то метод должен быть реализован в подклассах.
 */
abstract void HitTarget(Object target);

/*
 * Метод, реализующий движение пули.
 * Так как разные типы пуль имеют разную траекторию
 * движения, то метод должен быть реализован
 * в подклассах.
 */
abstract void Movement();
}

/*
 * Класс пули для автоматического оружия.
 */
class AutomaticBullet extends AbstractBullet
{
    public void HitTarget(Object target){
        // реализация поражения цели target
        System.out.println("Hit by automatic bullet\n");
    }
    public void Movement(){
        // реализация алгоритма движения пули
    }
}

```

```

/*
 * Класс пули для дробовика.
 */
class ShotgunBullet extends AbstractBullet
{
    public void HitTarget(Object target){
        // реализация поражения цели target
        System.out.println("Hit by shotgun bullet\n");
    }
    public void Movement(){
        // реализация алгоритма движения пули
    }
}

```

Реализация классов оружия представлена в листинге 3.3.2. Метод **Shoot()** этого класса реализует выстрел. Технология выстрела одинакова для всех типов оружия. Для создания пули, специфической для конкретного типа оружия, вызывается фабричный метод **CreateBullet()**, который реализуется в каждом из подклассов. Этот метод возвращает ссылку на **AbstractBullet**, под которой в каждом конкретном случае будет представлен объект соответствующего класса пули.

Листинг 3.3.2. Реализация классов оружия

```

/*
 * Класс абстрактного оружия
 */
abstract class AbstractWeapon {
    /*
     * Фабричный метод для создания пули.
     */
    protected abstract AbstractBullet CreateBullet();
}

```

```

private Point3D location;
private Vector3D direction;
private double caliber;
/*
 * Текущее положение оружия
 */
public Point3D GetLocation() {
    return location;
}
public void SetLocation(Point3D newLocation) {
    location = newLocation;
}
/*
 * Направление оружия
 */
public Vector3D GetDirection() {
    return direction;
}
public void SetDirection(Vector3D newDirection) {
    direction = newDirection;
}
/*
 * Калибр оружия
 */
public double GetCaliber() {
    return caliber;
}
public void SetCaliber(double newCaliber) {
    caliber = newCaliber;
}
/*
 * Метод, производящий выстрел.
 * Возвращает экземпляр созданной пули.
 */
public AbstractBullet Shoot() {
    // создание объекта пули с помощью фабричного
    // метода
    AbstractBullet bullet = CreateBullet();

```

```

        // настройка пули на текущие параметры оружия
        bullet.SetCaliber(this.GetCaliber());
        bullet.SetLocation(this.GetLocation());
        bullet.SetDirection(this.GetDirection());
        // начать движение пули
        bullet.StartMovement();
        // вернуть экземпляр пули
        return bullet;
    }
}

/*
 * Класс автоматического оружия.
 */
class AutomaticWeapon extends AbstractWeapon
{
    public AutomaticWeapon(){
        SetCaliber(20);
    }
    /*
     * Реализация фабричного метода.
     * Создает экземпляр пули,
     * специфический для текущего типа оружия.
     */
    protected AbstractBullet CreateBullet(){
        return new AutomaticBullet();
    }
}

/*
 * Класс дробовика.
 */
class Shotgun extends AbstractWeapon
{
    public Shotgun(){
        SetCaliber(50);
    }
}

```

```

/*
 * Реализация фабричного метода. Создает экземпляр
 * пули, специфический для текущего типа оружия.
 */
protected AbstractBullet CreateBullet() {
    return new ShotgunBullet();
}
}

```

Предложенный фрагмент реализации модели «оружие — пуля» демонстрирует только её архитектурные особенности и не претендует на функциональную полноту (в реальной игровой программе, скорее всего, технология выстрела выглядела бы намного сложнее).

Диаграмма классов описанной модели представлена на рис. 3.3.2.

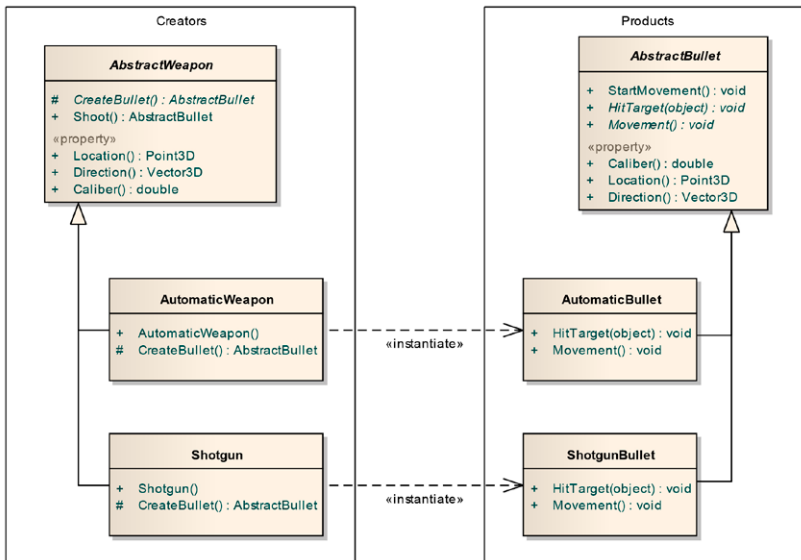


Рис. 3.3.8. Диаграмма классов модели «оружие — пуля»

Предложенный подход обладает такими преимуществами:

1. Риск того, что оружие останется без объекта пули в момент выстрела или что будут производиться пули не характерного оружию типа, сведен к минимуму, так как за создание пули отвечает само оружие;
2. Клиентский код, использующий оружие и пулю, не зависит от того, каким типом пули производится выстрел, ему достаточно только знать о классе `AbstractBullet`;
3. Ответственность за конфигурирование оружия конкретным типом производимых пуль ложится не на клиентский код, а на конкретный класс оружия, что позволяет упростить архитектуру клиента.

Недостатки предложенного решения:

1. Тип пуль, которыми стреляет конкретный вид оружия, определяется статически на этапе компиляции, его нельзя изменять на этапе выполнения, поэтому заданный тип оружия может производить только один вид пуль;
2. Для реализации возможности стрелять новыми пулями надо реализовать новый класс оружия, что, в конечном итоге, может привести к росту иерархии классов оружия.

К уроку приложена папка с примерами реализации паттернов. Порождающие паттерны расположены в папке `Creational`. Полный код примера находится в папке `FactoryMethod`.

3.4. Prototype

3.4.1. Название паттерна

Prototype/Прототип.

Описан в работе [GoF95].

3.4.2. Цель паттерна

Определяет виды создаваемых объектов с помощью экземпляра-прототипа и создает новые объекты путем копирования этого прототипа [GoF95].

3.4.3. Паттерн следует использовать, когда...

- Клиентский код должен создавать объекты, ничего не зная об их классе, или о том, какие данные они содержат.
- Классы создаваемых объектов определяются во время выполнения (например, при динамической загрузке).
- Экземпляры класса могут пребывать в не очень большом количестве состояний, поэтому может оказаться значительно удобнее создать несколько прототипов и клонировать их вместо прямого создания экземпляра класса.

3.4.4. Причины возникновения паттерна

Допустим, что перед нами стоит задача разработать игру «Тетрис», в частности, «Стройку». Для тех, кто не когда не играл эту замечательную игру, вкратце изложим суть. С верхней части игровой области падают строительные блоки разной формы. Задача игрока заключается в том, чтобы не позволить строительным блокам заполнить всю игровую область снизу доверху. Для этого надо стараться располагать блоки так, чтобы полностью

заполнить горизонтальную линию, так как полностью заполненные линии исчезают.

Набор разных форм строительных блоков фиксирован. Программа случайным образом выбирает форму последующего строительного блока, создает блок в соответствии с выбранной формой и начинает продвигать его по игровому полю.

Стоит следующий вопрос. Как создавать экземпляры строительных блоков, минимизировав при этом зависимость клиентского кода от типа формы блока?

Можно предложить следующее решение. Пусть каждый объект строительного блока умеет создавать копию самого себя через некий универсальный интерфейс. В программе есть список блоков-прототипов, каждый из которых представляет одну из возможных форм. При необходимости создания следующего блока его прототип случайным образом выбирается из списка, после чего создается новый экземпляр строительного блока посредством клонирования выбранного прототипа. Поскольку предусмотрен универсальный интерфейс для клонирования, то клиентский код полностью не зависит от того, строительный блок какой формы ему надо клонировать.

3.4.5. Структура паттерна

Общая структура паттерна представлена на рис. 3.4.9.

Участники паттерна:

- **Prototype** — прототип
 - ▷ Определяет интерфейс для клонирования самого себя (в языке C# для таких целей можно использовать стандартный интерфейс `ICloneable`).

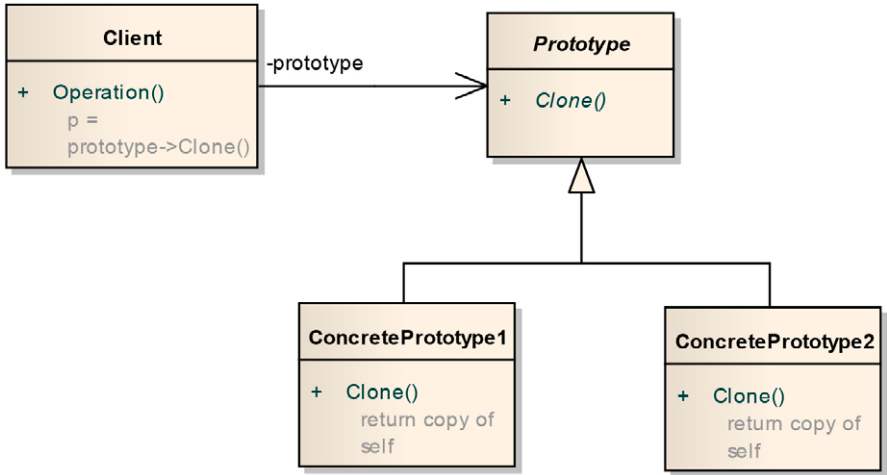


Рис. 3.4.9. Общая структура паттерна Prototype

- **ConcretePrototype** — конкретный прототип
 - ▷ Реализует операцию клонирования самого себя.
- **Client** — клиент
 - ▷ Создает новый объект, посылая запрос прототипу копировать самого себя.

Отношения между участниками:

- Клиентский код обращается к прототипу с просьбой создать копию самого себя.

3.4.6. Результаты использования паттерна

Как и все порождающие паттерны, прототип скрывает от клиента конкретные классы продуктов, уменьшая тем самым его сложность. Новые классы продуктов можно создавать практически без модификаций клиентского кода.

Добавление/удаление новых типов продуктов во время выполнения. Объекты-прототипы можно соз-

давать и удалять на этапе выполнения. Такое решение представляет большую гибкость по сравнению с **Abstract Factory** или **Factory Method**.

Определение новых типов продуктов без необходимости наследования. Для создания нового типа продукта надо определить его прототип, что, в общем случае, не требует наследования. Это, зачастую, позволяет избавиться от больших иерархий классов, которые требуются при использовании паттернов **Abstract Factory** или **Factory Method**.

Использование диспетчера прототипов. Зачастую для управления прототипами в системе используется специальный диспетчер, в котором регистрируются прототипы. Класс-диспетчер позволяет получать необходимый прототип за его именем, а также динамически добавлять или удалять прототипы.

Отсутствие параметров в методе `CloneIt()` обеспечивает наиболее общий интерфейс для клонирования.

Реализация метода `CloneIt()` — наиболее трудная часть при использовании паттерна. Особенности трудности проявляются тогда, когда надо клонировать объекты, которые он инкапсулирует, а также в случае наличия круговых ссылок. Зачастую, базовая реализация клонирования уже реализована на уровне основной библиотеке языка программирования (например, в C# или Java).

3.4.7. Практический пример использования паттерна

Давайте реализуем паттерн **Prototype** в практическом примере. Создадим иерархию наследования устройств. Базовым классом иерархии будет **Device**. Он определит

базовый интерфейс и главным в нем для нас будет определение абстрактного метода **Clone**. Мы будем создавать реализацию этой функции в потомках класса **Device**.

Листинг 3.4.1. Класс Device

```
/*
 * Это абстрактный базовый класс Device.
 * Он определяет метод CloneIt, которая составляет
 * основу паттерна Prototype
 * Обращаем ваше внимание, что можно было использовать
 * стандартный интерфейс Cloneable и его метод Clone
 * Однако в учебных целях мы решили реализовать
 * паттерн без их использования
 */
abstract class Device {
    // название устройства
    private String name;
    // конструктора
    public Device() {
        SetName("Unknown device");
    }
    public Device(String dname){
        SetName(dname);
    }
    // вспомогательные методы
    public String GetName(){
        return name;
    }
    public void SetName(String dname) {
        name = dname;
    }

    // абстрактный метод
    // Она будет использоваться для создания копий
    abstract Device CloneIt();
}
```

```
// отображение данных
public void Show(){
    System.out.println("\nName is\n"+
                        GetName() + "\n");
}
}
```

Следует обратить внимание, что метод `CloneIt()` возвращает ссылку типа `Device`. Это заставляет потомка создавать свою реализацию `CloneIt`. Теперь поговорим о потомке. У нас им будет класс `Car`. Он описывает машину. Мы включили в него лишь некоторые свойства машины. Главным для нас является реализация метода `CloneIt`. Она создаст копию машины, когда в этом возникнет необходимость

Листинг 3.4.2. Класс Car

```
class Car extends Device {
    // свойства машины
    private String manufacturer;
    private String description;
    private String color;

    private int year;
    // конструктора
    public Car(){
        this("No information","No description",
            "No color",0);
    }
    Car(String cmanufacturer, String cdescription,
        String ccolor, int cyear){
        SetName("Car");
        SetManufacturer(cmanufacturer);
```

```

        SetDescription(cdescription);
        SetColor(ccolor);
        SetYear(cyear);
    }
    // вспомогательные функции
    public int GetYear(){
        return year;
    }
    public String GetManufacturer(){
        return manufacturer;
    }
    public String GetDescription(){
        return description;
    }
    public String GetColor(){
        return color;
    }
    public void SetYear(int cyear){
        year = cyear;
    }
    public void SetManufacturer(String cmanufacturer) {
        manufacturer = cmanufacturer;
    }
    public void SetColor(String ccolor) {
        color = ccolor;
    }
    public void SetDescription(String cdescription) {
        description = cdescription;
    }

    // реализация абстрактного метода в потомке
    public Device CloneIt() {
        Car c = new Car(this.GetManufacturer(),
                        this.GetDescription(),
                        this.GetColor(),
                        this.GetYear());
    }

```

```

        return c;
    }
    public void Show() {
        super.Show();

        System.out.println("\nDescription of car is\n" +
            GetDescription() + "\n");
        System.out.println("\nManufacturer of car is\n" +
            GetManufacturer() + "\n");
        System.out.println("\nYear of car is\n" +
            GetYear() + "\n");
        System.out.println("\nColor of car is\n" +
            GetColor() + "\n");
    }
}

```

Листинг 3.4.5. Клиентский код, использующий классы с прототипированием

```

import java.util.Scanner;
public class Run {
    public static void main(String[] args) {
        try {
            Scanner in = new Scanner(System.in);

            // Введем данные
            String manufacturer;
            System.out.println("\nInput manufacturer
                                of car:\n");
            manufacturer = in.nextLine();
            String description;
            System.out.println("Input description
                                of car:\n");
            description = in.nextLine();
            String color;
            System.out.println("Input color
                                of car:\n");

```

```

        color = in.nextLine();
        int year;
        System.out.println("Input year of car:\n");
        year = in.nextInt();
        // создадим объект
        Car c = new Car(manufacturer, description,
                        color, year);

        c.Show();
        System.out.println("Let's clone!\nLet's
                           prototype!\n");

        // клонируем объект
        Car copy = (Car)c.CloneIt();
        copy.Show();
    }
    catch(Exception ex) {
        System.out.println("Exception happened!
                           Exception description\n" +
                           ex.getMessage());
    }
}

```

Суть кода из листинга 3.4.5 такова:

1. Вводим данные о машине;
2. Создаем объект машины;
3. Создаем копию машину с помощью клонирования и вызова метода `CloneIt`.

Пример показывает, что реализация паттерна **Prototype** не слишком сложна по сравнению с рядом других паттернов.

*К уроку приложена папка с примерами реализации паттернов. Порождающие паттерны расположены в папке **Creational**. Название папки проекта **Prototype**.*

3.5. Singleton

3.5.1. Название паттерна

Singleton/Одиночка.

Описан в работе [GoF95].

3.5.2. Цель паттерна

Гарантирует, что у класса есть только один экземпляр, и предоставляет единую точку доступа к нему [GoF95].

3.5.3. Паттерн следует использовать, когда...

- Должен существовать только один экземпляр заданного класса, доступный всему клиентскому коду [GoF95].

3.5.4. Причины возникновения паттерна

Часто в программировании случается ситуация, когда надо определить некоторую переменную, которая доступна глобально и может существовать только в одном экземпляре. Например, для доступа в систему бухгалтерского учета организации пользователю необходимо пройти аутентификацию, указав свое имя и пароль. После успешной аутентификации пользователь получает доступ к системе. При этом в каждый момент времени системе должна быть известны персональные данные пользователя: его полное имя, уровень доступа и, возможно, другие. Для реализации такой возможности следует определить некоторый объект, который доступный из каждой точки системы (например, для определения того, есть ли доступ в пользователя к определенным возможностям системы) и может существо-

вать только в одном экземпляре (так как одновременно в системе не могут аутентифицироваться два и больше пользователей).

В языке C++ это можно реализовать посредством использования глобальной переменной или статического поля класса. Но такой подход имеет недостатки:

1. Клиентский код имеет свободный доступ к этой переменной, что может привести к её несанкционированным изменениям;
2. В клиентском коде существует потенциальная возможность создания более одного экземпляра такой переменной.

Следует отметить, что в полностью объектно-ориентированных языках, какими являются C# и Java, вообще исключена возможность определения глобальных переменных, поэтому в таких языках для определения глобального состояния можно использовать только статические поля класса.

Для преодоления недостатков представленной идеи воспользуемся таким подходом:

1. Возможность создания собственного экземпляра должен иметь только сам класс. В таком случае легко проконтролировать количество созданных экземпляров и, при необходимости, ограничить его одним объектом.
2. Доступ к своему единственному экземпляру также должен предоставлять сам класс.
3. Предусмотреть возможность модификации поведения объекта-одиночки через наследование так, что

при подмене единственного экземпляра объектом его подкласса, клиентский код не должен требовать модификаций.

Техническая реализация предложенного подхода может выглядеть так:

1. Для запрета создания экземпляров класса внешним кодом его конструктор (или конструкторы) определяется как **protected**.
2. Доступ к экземпляру класса осуществляется посредством его публичного статического метода.
3. Члены класса единственного экземпляра (кроме переменной, в которой сохраняется ссылка на единственный объект, и метода предоставления доступа к этой переменной) должны быть нестатическими. Это позволит, при надобности, модифицировать поведение класса, наследовав от него подклассы.

3.5.5. Структура паттерна

Общая структура паттерна представлена на диаграмме 3.5.1.

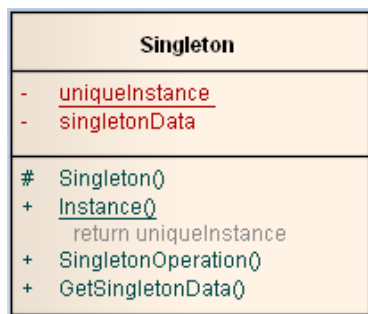


Рис. 3.5.10. Общая структура паттерна Singleton

Участники паттерна:

- **Singleton** — одиночка

- ▷ Обеспечивает создание только одного экземпляра самого себя, ссылка на который сохраняется в статической переменной `uniqueInstance`, и глобальный доступ к нему через статический метод `Instance()`.
- ▷ Запрещает клиентскому коду создавать собственные экземпляры, запретив ему доступ к своему конструктору (конструктор одиночки определяется как защищенный или приватный).

Отношения:

- Клиентский код имеет возможность доступа к экземпляру `Singleton` только через его статический метод `Instance()`.

3.5.6. Результаты использования паттерна

Есть возможность полного контроля доступа клиента к единственному экземпляру. Что, например, может дать возможность подсчитать количество клиентских обращений или запрещать доступ в случае отсутствия соответствующих прав.

Единственные экземпляры объекта обладают большими возможностями, нежели обычные глобальные переменные.

Допускается уточнение поведения посредством наследования.

Допускается возможность создания более одного экземпляра. Для этого достаточно модифицировать метод `Instance()`.

Использование одиночки представляет более гибкие возможности, нежели статические функции классов или статические классы C#, так как такие программные конструкции, как правило, обладают рядом ограничений, делающих не возможным определение виртуальных функций с последующим замещением их в подклассах.

Использование глобальных переменных и одиночек способствует созданию «лгущих» интерфейсов, т.е. интерфейсов, которые не показывают явных зависимостей с другими необходимыми для его функционирования объектами.

При реализации одиночки в многопоточной среде следует также учитывать тот факт, что два параллельных потока могут одновременно получать доступ к методу создания единственного экземпляра, что может привести к созданию более одного экземпляра одиночки. Во избежание ситуации «борьбы за ресурсы», следует код, ответственный за создание объекта, поместить в критическую секцию, предоставив тем самым доступ к нему только одному потоку.

3.5.7. Практический пример использования паттерна

Для каждой программы полезно вести специальный журнал, в котором фиксируются определенные события. В такой журнал, например, можно записывать все исключительные ситуации, возникшие в процессе работы. Потом такие записи можно тщательно проанализировать и обнаружить проблемные или дефектные части системы. Или, в случае жалоб пользователей, можно определить характер и причину сбоев. Подобного рода журналы так-

же часто используются для вывода сообщений в процессе отладки программы.

В нашем случае к программному журналу выставляются следующие требования:

1. Сообщения журнала должны выводиться в специальный текстовый файл.
2. В программе должен присутствовать только один экземпляр журнала.

Сразу обращаем ваше внимание, что наш пример синглтона не является многопоточным. Это означает, что, если вы в своей программе оперируете несколькими потоками ваш класс синглтона может порождать неожиданные ошибки. Поэтому, когда вы познакомитесь с понятием потока и объекта синхронизации вам нужно будет переписать класс синглтона с учетом полученных знаний.

Для реализации класса журнала, назовем его `Logger`, воспользуемся идеей паттерна `Singleton`.

В классе `Logger` объявим статическую переменную-член типа ссылка на `Logger` с названием `refInstance`, в которой будем хранить адрес единственного объекта. Для доступа к этой переменной из внешнего кода создадим статический метод `GetInstance`. Задачей этой функции является возврат ссылки на единственный объект типа `Logger`. В случае равенства `refInstance null` мы выделяем память для единственной копии `Logger`. Таким способом реализуется создание журнала по требованию: если в процессе работы программы не было обращений к журналу, то его экземпляр не создается.

Для исключения возможности создания более одного экземпляра **Logger** в процессе работы программы возможность создавать его объект должен иметь только сам класс **Logger**. С этой целью единственный конструктор класса объявляется закрытым.

Полная реализация класса **Logger** представлена в листинге 3.5.1.

Листинг 3.5.1. Реализация класса Logger

```
/*
 * Класс журнала событий программы.
 * Предназначение - запись событий в специальный
 * текстовый файл.
 * В программе может существовать только в одном
 * экземпляре.
 */
class Logger {
    // конструктор закрыт, чтобы нельзя было создать
    // копию объекта в обход специального метода

    private Logger() {}
    // ссылка на будущий объект логера
    static Logger refInstance;

    // статический метод для получения доступа
    // к объекту логирования
    // Объект создается, если он не существовал.
    // Если он существовал возвращается ссылка на
    // уже созданный объект
    public static Logger GetInstance() {
        if (refInstance == null) {
            refInstance = new Logger();
        }
        return refInstance;
    }
}
```

```
// метод для записи строк в лог-файл
public void PutMessage(String message)
    throws FileNotFoundException{
    try(PrintStream out =
        new PrintStream(new FileOutputStream(
            "logsingleton.txt",true))) {
        out.print(message+"\n"+LocalDateTime.
            now().toString()+"\n");
    }
}
}
```

*К уроку приложена папка с примерами реализации паттернов. Порождающие паттерны расположены в папке *Creational*. Полный код примера находится в папке *Singleton*.*

3.6. Анализ и сравнение порождающих паттернов

Есть два наиболее распространенных способа параметризации системы классами объектов, которые создаются.

Первый способ заключается в применении наследования: для создания нового объекта создается новый класс. Такой подход соответствует паттерну **Factory Method**. Как уже говорилось ранее, основными недостатками этого подхода являются статическое определение классов создаваемых объектов и опасность создания большого количества классов, которые ничего не делают, кроме как реализуют фабричный метод, посредством которого создаются объекты.

Второй способ основывается на композиции: определяется объект, которому известно о классах объек-

тов-продуктов. Этот объект делается параметром системы и может изменяться на этапе выполнения. Такой подход характерный для паттернов **Abstract Factory**, **Builder**, **Prototype**. Каждый из этих паттернов создает специальный «фабричный объект», который создает продукты. В случае **Abstract Factory** такой объект отвечает за создание семейства продуктов. При использовании **Builder** фабричный объект отвечает за пошаговое создание сложного продукта. Паттерн **Prototype** предусматривает объект, который умеет клонировать сам себя.

Несколько обособленно среди других порождающих паттернов стоит **Singleton**. Его предназначение — создание глобального объекта-одиночки. Часто абстрактная фабрика реализуется как одиночка, если нет смысла создавать для нее более одного экземпляра. **Singleton** также может использоваться при реализации паттерна **Prototype**. Пример этому — реализация палитры конфигураций, описанная в пункте 3.4.7.

Поскольку **Singleton** представляет собой аналог глобальной переменной, а работать с глобальными переменными всегда надо с особой осторожностью, то в современном программировании его относят к антипаттернам и стараются по возможности не использовать.

4. Домашнее задание

1. Спроектировать универсальный каркас многодокументного редактора. Редактор должен представлять основные функции работы с документом:
 - ▷ Создание.
 - ▷ Открытие.
 - ▷ Сохранение.
 - ▷ Сохранение под новым именем.
 - ▷ Печать.
 - ▷ Закрытие.

Предложенный объектно-ориентированный дизайн каркаса редактора должен без изменений использоваться для разработки редакторов документов различных типов.

2. На основании каркаса, разработанного в задаче 1, спроектировать редактор, предназначенный для работы с текстовыми документами.
3. На основании каркаса, разработанного в задачи 1, спроектировать редактор, предназначенный для работы с графическими документами различных форматов. Редактор обязательно должен иметь возможность сохранять изображение в выбранном графическом формате, а также иметь палитру инструментов для обработки изображения.

5. Использованные информационные источники

- [GoF95] Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — СПб: Питер, 2001. — 386 с.
- [Grand2004] Гранд М. Шаблоны проектирования в Java / М. Гранд; Пер. с англ. С. Беликовой. — М.: Новое знание, 2004. — 559 с.
- [SM2002] Стелтинг С., Маасен О. Применение шаблонов Java. Библиотека профессионала.: Пер. с англ. — М.: Издательский дом «Вильямс», 2002. — 576 с.
- [DPWiki] [Шаблоны проектирования \(Wikipedia\)](#)
- [DPOverview] [Обзор паттернов проектирования](#)

Урок №2.
Структурные паттерны

1. Понятие структурного паттерна

Исходя из определения информационных систем, является очевидным, что всякое программное решение, так ли иначе, оперирует некоторой совокупностью данных, при анализе которых мы получаем некоторую информацию, позволяющую нам осуществлять принятие бизнес решений и штатных решений по управлению самой информационной системой.

Анализ данных предполагает, что они должны быть организованы в некоторой структуре, которая позволит с одной стороны идентифицировать различные данные (логически отличать их друг от друга), а с другой — организовывать атомарные величины в структурированные объекты и их совокупности. Структурирование данных необходимо для отражения понятий, которыми мы оперируем при анализе, а также отображения отношений, которые образуются между объектами в процессе их появления.

Объектно-ориентированный подход предоставляет нам богатый инструментарий в смысле описания структурированных понятий и отражения отношений, в которых они состоят. Однако организация структуры, связывающей понятия в работающую систему, имеет решающее значение с точки зрения эффективного применения информационной системы, а также с точки зрения сопровождения, полученного в результате процесса разработки продукта.

Паттерны проектирования в общем смысле отражают наилучшие модели решения прикладных проблем, прошедшие проверку временем и практикой применения. Структурные паттерны предлагают модели организации данных в структуры, которые наилучшим образом решают вопрос организации управления данными, которые мы используем в своих прикладных решениях. Следует понимать, что данными можно считать не только атомарные величины, но и все объекты, существующие в пределах нашего приложения.

Далее последует перечисление основных структурных паттернов, признанных мировым сообществом лучшей практикой разработки программного обеспечения.

2. Паттерн Adapter

2.1. Цель паттерна

Для лучшего понимания предлагаемого материала, а также с целью упрощения изложения, нами будет введена следующая терминология, специфичная рассматриваемому вопросу: под клиентом (*client*) мы будем понимать некоторый класс, который использует (в общем случае агрегирует) некоторый класс, который мы называем **адаптируемым** (*adaptee*). Под **адаптером** (*adapter*) мы будем понимать класс, выполняющий приведение интерфейса адаптируемого класса к интерфейсу, ожидаемому клиентом.

Цель паттерна проектирования **Adapter** (англ. «*adapter*») состоит в том, чтобы привести (адаптировать) интерфейс некоторого адаптируемого класса к интерфейсу, который ожидается клиентом.

2.2. Причина возникновения паттерна

Достаточно часто встречается следующая проблема: у нас в наборе и инструментов имеется некоторый класс, который мы хотим использовать в неспецифичной для его структуры задаче. Например, у нас объявлен тип данных, описывающий понятие сетевого устройства и названный нами **IPEndPoint**, и наделённый такими свойствами как IP-адрес, мак-адрес и имя хоста, которые мы используем в целях некоторого прикладного анализа (например, трассировки перемещения пакетов). Анализ выполняется некоторым классом, который агрегирует множество объектов типа **IPEndPoint** и называется **NetView**. Однако

мы хотим реализовать графическое представление для процесса и результата анализа, с выводом его на основное окно нашего приложения. Проблема состоит в том, что класс `NetView` не имеет интерфейса, специфичного для объекта графической подсистемы и, соответственно, не может быть использован оконным классом для выполнения прорисовки. Мы не имеем возможности «переписать» (изменить исходный текст и соответственно структуру) класс `NetView`, под нужды приложения, поскольку он является частью, используемого нами набора типов из dll-библиотеки предоставленной сторонними разработчиками; или мы просто не хотим «перегружать» структуру класса, поскольку она критична для каких-либо задач нашего приложения.

В таком случае логично использовать некоторый класс-посредник, который, используя наследование, позволит привести класс `NetView` к необходимому типу данных, а также посредством переопределения методов, специфичных для компонента графической подсистемы, определит интерфейс прорисовки типа данных `NetView`.

Такой класс-посредник обычно называют адаптером.

2.3. Структура паттерна

Структура паттерна `Adapter` представлена на приведённой слева диаграмме.

Участвующие элементы:

- **Client** — класс, который использует некоторые вспомогательные типы данных и ожидает, что они имеют стандартный интерфейс взаимодействия (использования) описанный классом `Target`.

- **Target** — класс, имеющий интерфейс, ожидаемый клиентом.
- **Adaptee** — класс, который необходим для работы клиента, но имеет интерфейс, отличный от того, который ожидается клиентом.
- **Adapter** — класс, выполняющий приведение интерфейса класса **Adaptee**, к интерфейсу класса **Target**.

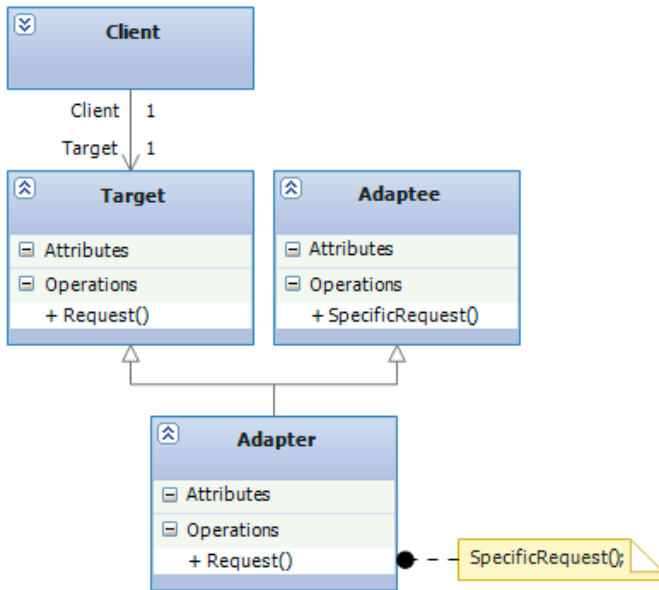


Рис. 2.3.1.

В, предложенной выше структуре, приведение интерфейса выполняется за счёт того, что класс **Adapter** наследует оба класса **Adaptee** и **Target**, а, значит, обладает интерфейсами обоих этих классов. Затем класс **Adapter** приводит вызовы методов специфичных для интерфейса класса **Target** к вызовам соответствующих методов интерфейса класса **Adaptee**.

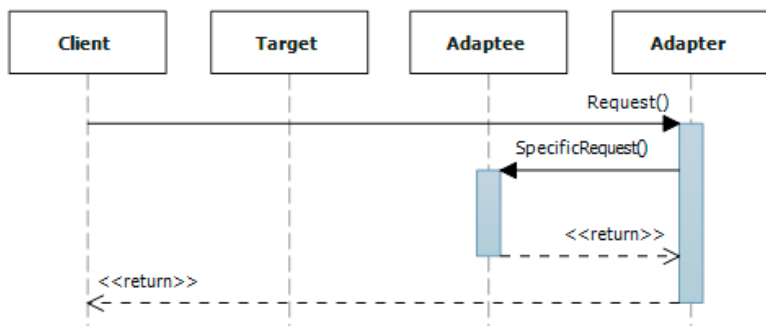


Рис. 2.3.2.

На представленной ниже диаграмме последовательности демонстрируется, что вызовы методов объекта класса **Adapter** сводятся к вызовам методов объекта его базового класса.

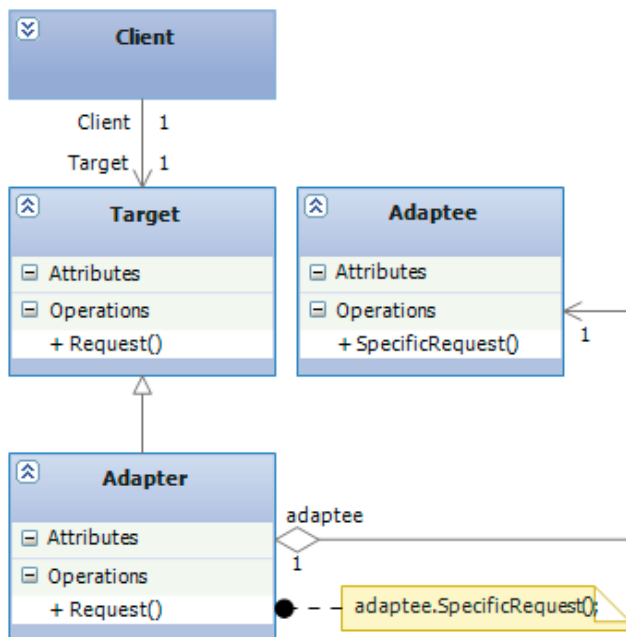


Рис. 2.3.3.

Паттерн **Adapter** также может быть реализован альтернативным способом, структура которого представлена на приведённой слева диаграмме.

Отличие от предыдущей структуры состоит в том, что в текущей модели классы **Adapter** и **Adaptee** находятся не в отношении родства, а в отношении ассоциации, то есть класс **Adapter** агрегирует класс **Adaptee**.

Таким образом, приведение интерфейса класса **Adaptee** к интерфейсу класса **Target** выполняется за счёт того, что вызовы методов объекта класса **Adapter**, специфичные для интерфейса класса **Target** приводятся к вызовам соответствующих методов объекта класса **Adaptee**, инкапсулированного в классе **Adapter**. Нижеприведённая диаграмма последовательности иллюстрирует происходящее.

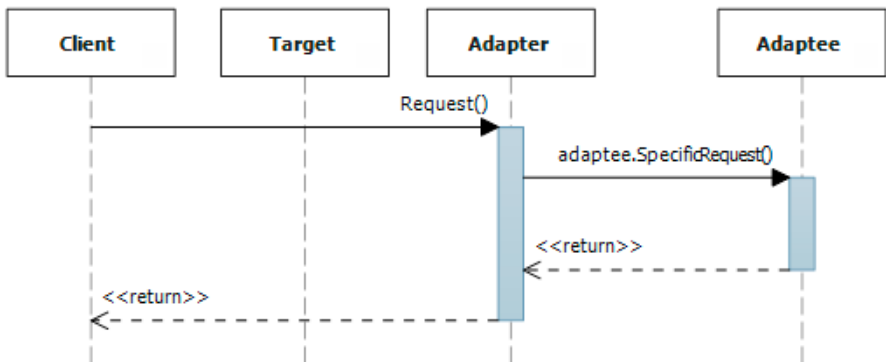


Рис. 2.3.4.

2.4. Результаты использования паттерна

Основной положительный результат использования паттерна проектирования **Adapter** состоит в том, что мы получаем возможность гибко привести интерфейс неко-

торого класса к интерфейсу, ожидаемому приложением без изменения структуры самого класса. Это необходимо, как с точки зрения устранения избыточности структуры типов, так и с точки зрения модульности создаваемых приложений.

Избыточность играет отрицательную роль тогда, когда нам необходимо повторно использовать написанный нами код (например, в другом приложении) Такой код называется *reusable*-кодом. Создание *reusable*-кода считается хорошей практикой, поскольку уменьшает стоимость и увеличивает скорость разработки. А также увеличивает гибкость и масштабируемость создаваемых приложений за счёт модульной структуры готового приложения.

Расширять приложение путём добавления новых типов значительно проще, чем полностью заново создавать некоторые модули.

2.5. Практический пример использования паттерна

Мы рассмотрим использование паттерна *Adapter* на примере приложения, выполняющего управление товарооборотом некоторого предприятия. Для осуществления управления товарами мы опишем тип данных *Product* (продукт/товар), который будет организован в коллекцию товаров при помощи класса *Products Collection*.

Наше приложение так же выполняет управление и другими аспектами работы торгового предприятия, которые мы «опустим» для прозрачности примера.

Для реализации возможности создавать переносимую резервную копию данных, которую можно использовать для практически любых задач мы решили сделать так,

чтобы можно было все данные экспортировать в отдельные xml-файлы. Для этого мы реализовали класс `XmlIO`, которые осуществляет запись и чтение Xml-документа, и объект которого инкапсулируется оконным классом нашего приложения. Для реализации записи в файл коллекции продуктов нами был создан класс `ProductXmlIOAdapter`, который инкапсулирует коллекцию продуктов, реализует приведение этой коллекции к xml-документу, а также xml-документа к коллекции.

Таким образом, мы реализуем функцию записи коллекции элементов в файл в xml-формате и в то же время не изменяем структуры исходного типа данных и устраняем избыточность структуры, которая могла появиться вследствие наполнения класса `Product` дополнительным функционалом. Устранение избыточности необходимо постольку, поскольку тип данных `Product` может использоваться при приведении информации о продуктах, полученной из некоторой базы данных, к объектному представлению, для выполнения последующего анализа и так далее. При выполнении всех этих действий избыточность структуры типа может создавать дополнительные сложности. Также, подобная модульная структура добавляет гибкости при сопровождении проекта и использовании (создании) reusable-кода.

Далее представлена диаграмма классов, иллюстрирующая описанное выше приложение.

К уроку приложена папка с примерами реализации паттернов. Структурные паттерны расположены в папке Structural. Реализация паттерна Adapter находится в папке Adapter.

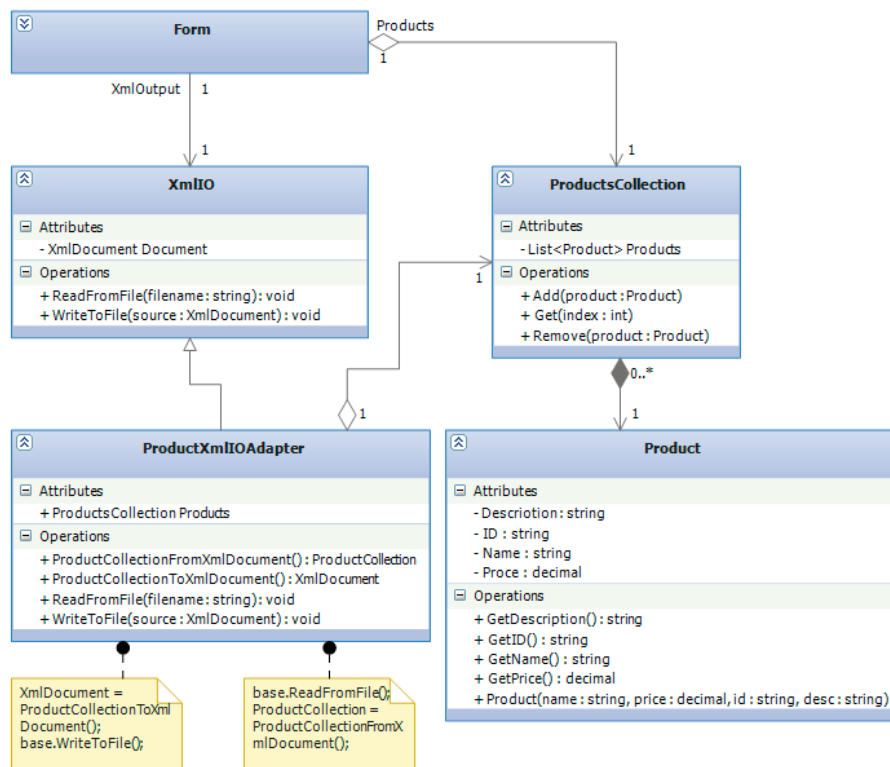


Рис. 2.5.5.

3. Паттерн Bridge

3.1. Цель паттерна

Цель паттерна **Bridge** (англ. «мост») состоит в том, чтобы отделить абстракцию от её реализации для того, чтобы они могли изменяться независимо друг от друга.

3.2. Причины возникновения паттерна

Обычно, в случаях, когда некоторая абстракция (обычно абстрактный класс) может иметь несколько конкретных реализаций, используют наследование для определения множества классов, с похожим (в общем случае говорят одинаковым или совместимым) интерфейсом. Абстрактный класс определяет интерфейс для своих потомков, который они реализуют «различными» способами.

Однако такой подход является не всегда достаточно гибким и имеет некоторые слабые стороны, способные привести к избыточности кода, а также создать дополнительные трудности при сопровождении проекта, что значительно увеличит его стоимость. Прямое наследование интерфейса абстракции некоторым конкретным классом связывает реализацию с абстракцией напрямую, что создаёт трудности при дальнейшей модификации реализации (её расширении), а также не позволяет повторно использовать абстракцию и её реализацию отдельно друг от друга. Реализация, как бы, становится «жёстко связанной» с абстракцией.

Паттерн проектирования мост предполагает помещение интерфейса и его реализации в различных иерар-

хиях, что позволяет отделить интерфейс от реализации и использовать их независимо, а также комбинировать любые варианты реализации с различными уточнёнными вариантами абстракции.

3.3. Структура паттерна

Паттерн проектирования мост представлен следующими структурными элементами:

- **Abstraction** (*абстракция*) — определяет интерфейс абстракции, а также содержит объект исполнителя, который определяет интерфейс реализации.
- **Implementor** (*исполнитель*) — определяет интерфейс для классов реализации. Интерфейс исполнителя не обязательно должен соответствовать интерфейсу абстракции. В принципе, интерфейсы, определённые абстракцией и исполнителем, могут быть совершенно разными, что является достаточно гибким. В целом, исполнитель должен определять базовые операции, на которых впоследствии базируется высокоуровневая логика абстракции.
- **RefinedAbstraction** (*уточнённая абстракция*) — расширяет интерфейс определённый абстракцией.
- **ConcreteImplementor** (*конкретизированный исполнитель*) — класс, который реализует интерфейс исполнителя и определяет его частную реализацию.

Абстракция и исполнитель совместно образуют «мост», который связывает уточнённую абстракцию с конкретной реализацией.

Структура паттерна проектирования мост представлена на рис. 3.3.1 в виде диаграммы классов.

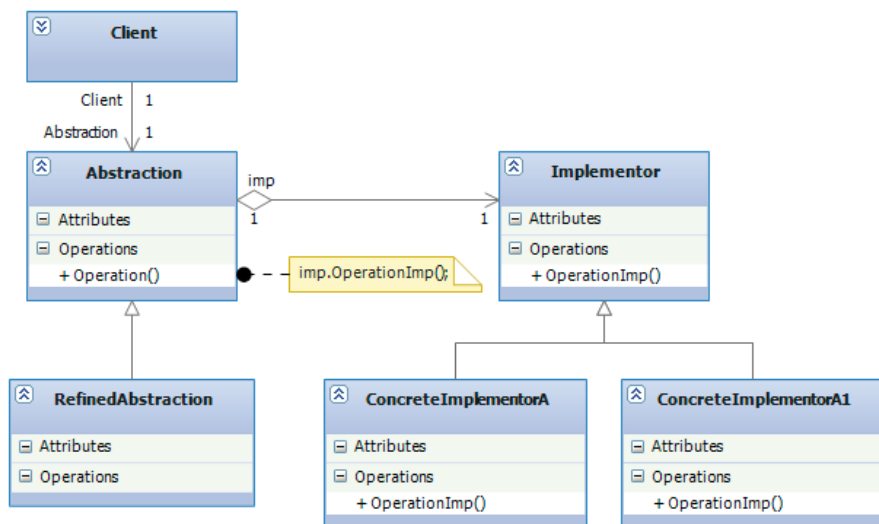


Рис. 3.3.1.

3.4. Результаты использования паттерна

Основное преимущество, которое предоставляет использование паттерна проектирования мост состоит в том, что выполняется логическое и структурное разделение абстракции от её реализации, что делает код более гибким. Так же применение паттерна мост улучшает такое качество кода, как расширяемость, поскольку абстракция и исполнитель находятся в различных иерархических структурах, а значит становится возможным расширение реализации независимо от абстракции, и наоборот.

Ещё одной причиной в пользу применения паттерна проектирования мост служит тот факт, что он позволяет скрывать детали реализации от клиента (*client*), то есть

от приложения, которое использует абстракцию, что позволяет клиенту быть независимым от того, какая именно реализация была выбрана в том или ином случае.

3.5. Практический пример использования паттерна

Мы рассмотрим применение паттерна проектирования мост на примере «смешанного» графического редактора (редактора, позволяющего совместно, в рамках одного представления, редактировать растровую и векторную графику).

Модель приложения предполагает наличие некоторого графического представления, оперирующего некоторыми абстрактными фигурами, интерфейс взаимодействия с которыми описывается интерфейсом `Figure`, играющим роль абстракции в данном примере.

Интерфейс исполнителя определяется интерфейсом `FigureImp`, от которого мы наследуем классы `VectorFigure` и `RasterFigure` — соответственно, описывающих реализации прорисовки векторной и растровой фигур.

После определения реализации мы можем уточнить абстракцию, описав интерфейса некоторой конкретной фигуры (например, эллипса). После этого можно связать уточнённую абстракцию с конкретной реализацией, например, определив классы `VectorEllipse` и `RasterEllipse`, описывающие соответственно векторный и растровый эллипсы. Модель описанного выше приложения представлена на рис. 3.5.2.

К уроку приложена папка с примерами реализации паттернов. Структурные паттерны расположены в папке `Structural`, реализация паттерна `Bridge` в папке `Bridge`.

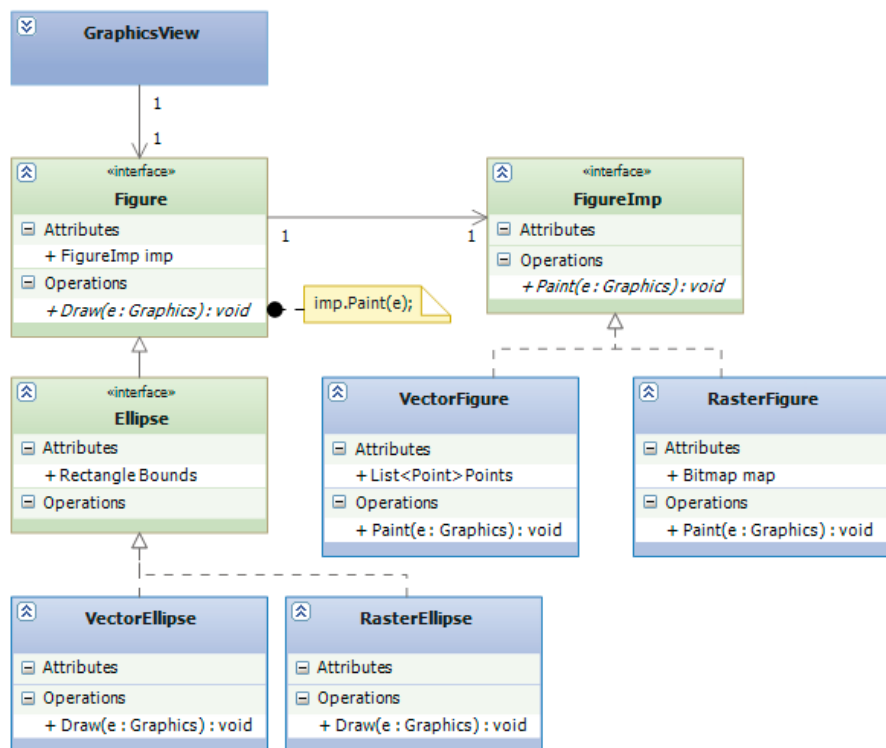


Рис. 3.5.2.

4. Паттерн Composite

4.1. Цель паттерна

Паттерн **Composite** (компоновщик) предназначен для того, чтобы представить объекты в виде структуры дерева в иерархической связи часть-целое.

4.2. Причины возникновения паттерна

Паттерн компоновщик существует потому, что структура типа «дерево» является достаточно распространённой и часто используется для организации данных, имеющих регулярную структуру. То есть такую структуру, которая реализует иерархическую зависимость часть-целое, предполагающую, что элементом некоторой композиции данных может быть не только элементарный элемент, но и такая же композиция.

Самая простая реализация подобной структуры с рекурсивной системой вложенности может быть выражена в виде некоторой системы типов, содержащей классы, описывающие элементарные компоненты, и классов, которые будут использоваться в качестве контейнеров для элементарных компонент.

Но подобный подход имеет серьёзный недостаток, состоящий в том, что код, использующий указанные классы, должен отдельно обрабатывать элементы и их контейнеры, даже если в большинстве случаев они обрабатываются одинаково. Это резко усложняет реализацию приложения.

Паттерн компоновщик предлагает рекурсивную структуру, при которой не придётся принимать решения о том,

как обрабатывать отдельные элементы общей совокупности данных.

Ключ к пониманию природы компоновщика состоит в определении абстрактного идентичного интерфейса для элементарных компонент и их контейнеров. Таким образом, поскольку контейнеры и элементарные компоненты находятся в отношении родства и имеют общий пользовательский интерфейс, то контейнер может быть элементом другого такого же контейнера, что создаёт удобную регулярную (рекурсивную) структуру.

4.3. Структура паттерна

- **Component** (*компонент*) — описывает интерфейс для объектов и их композиций; реализует базовое поведение, специфичное и для отдельных элементов и для их композиций; определяет интерфейс доступа к элементам композиции и управления этими элементами, а также определяет интерфейс доступа к родительскому элементу рекурсивной структуры.
- **Leaf** (*лист*) — определяет отдельный элемент композиции и описывает поведение «примитивных» (базовых) элементов общей структуры.
- **Composition** (*композиция*) — определяет поведение для компонентов, содержащих дочерние элементы, инкапсулирует дочерние элементы, а также реализует операции управления дочерними элементами и доступа к ним, определённые интерфейсом компонента.
- **Client** (*клиент*) — управляет элементами композиции через интерфейс компонента.

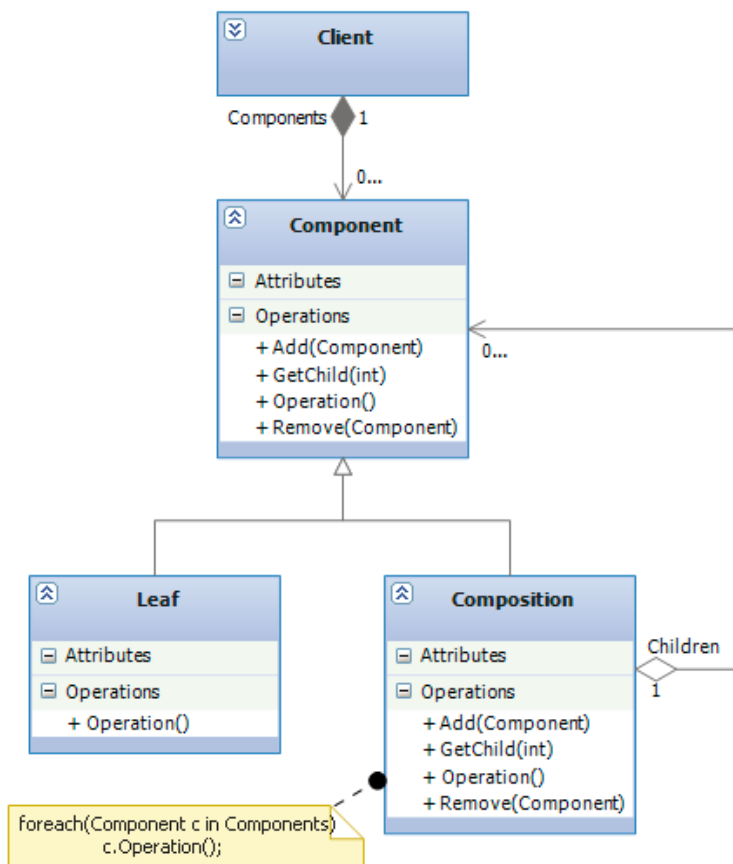


Рис. 4.3.1.

Структура паттерна компоновщик представлена в виде диаграммы классов, приведённой на рисунке 4.3.1.

4.4. Результаты использования паттерна

Использование паттерна компоновщик помогает упростить организацию элементов в виде вложенной структуры данных и устраняет избыточность кода при реализации этой задачи. Также компоновщик делает кли-

ентский объект более простым и позволяет ему обрабатывать элементарные компоненты и их композиции одинаково, в силу их унифицированного интерфейса, определённого классом-компонентом.

Компоновщик упрощает процесс добавления новых компонент. Новые классы, независимо от того, являются они элементарными компонентами или композициями, будут работать с уже существующей на момент их создания структурой. Другими словами, нет необходимости изменять клиента при создании новых компонент.

4.5. Практический пример использования паттерна

Мы рассмотрим применение паттерна проектирования компоновщик на базе организации множества элементов графического интерфейса пользователя, которое имеет регулярную структуру. Другими словами, всякое окно может содержать элементы управления, но также и другие окна. Окна можно упрощённо определить как композиции элементов управления.

Для организации компоновщика под определённую нами задачу мы объявляем абстрактный класс `UIElement`, описывающий некоторый элемент графического интерфейса пользователя и определяющий интерфейс базового поведения элемента графического интерфейса и интерфейс управления дочерними элементами композиции.

К базовому поведению элемента графического интерфейса относятся методы `Hide`, `Paint` и `Show`, которые соответственно реализуют возможность показывать элемент, перерисовывать его графическое представление и прятать.

Для управления-доступа к дочерним элементам композиции определяются методы **Add**, **GetChild** и **Remove**, которые соответственно позволяют добавлять новый дочерний элемент, получить его и удалить элемент из композиции.

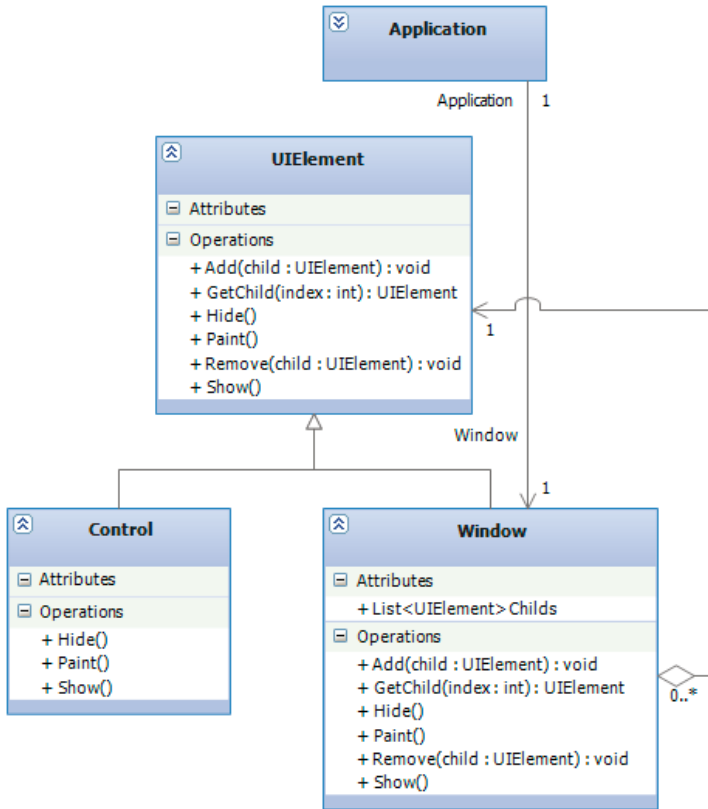


Рис. 4.5.2.

Класс **Control**, описывает элемент управления, который играет роль «примитивного» элемента реализуемого нами компоновщика и реализует базовое поведение. Класс **Window** представляет собой композицию элемен-

тов управления, но также реализует и базовое поведение, поскольку имеет некоторое элементарное графическое представление.

Класс `Application` играет роль клиента, который ассоциирован с некоторым окном, то есть использует объект класса `Window` и интерфейс, описанный классом `UIElement`.

Модель описанного приложения представлена на диаграмме классов на рисунке 4.5.2.

К уроку приложена папка с примерами реализации паттернов. Структурные паттерны расположены в папке `Structural`. Реализация паттерна `Composite` находится в папке `Composite`.

5. Паттерн Decorator

5.1. Цель паттерна

Цель паттерна **Decorator** (англ. «декоратор») состоит в том, чтобы реализовать возможность динамического добавления функционала к объекту, а также распределить ответственность за выполнение отдельных функций между отдельными классами.

Так же декоратор представляет собой альтернативу наследованию в смысле расширения функционала объектов.

5.2. Причины возникновения паттерна

Достаточно распространённым подходом является разделение ответственности за выполнение отдельных операций между отдельными классами, поскольку это создаёт структуру, при которой каждый класс инкапсулирует логику управления только теми обязанностями, которые на него возложены. И значение имеет не только модульность, упрощающая разработку, но и возможность закрывать классу доступ к операциям, которыми он управлять не должен по его сути.

Одним из методов, позволяющим распределить обязанности по выполнению некоторой общей задачи между отдельными классами является наследование. Однако такой подход является негибким, поскольку при наследовании добавленная функция статически закрепляется для всех потомков этого класса. И для того, чтобы создавать различные типы (с разным набором функциональности),

необходимо выполнять наследование «во всех возможных» комбинациях.

Декоратор же позволяет не только гибко комбинировать функциональность объекта по его внутренней логике, но и динамически изменять набор функций объекта во время выполнения, поскольку добавление функции сводится к созданию компонентного объекта необходимого класса.

5.3. Структура паттерна

Структура паттерна **Decorator** представлена следующими элементами:

- **Component** — представляет собой абстракцию некоторого элемента, для которого будут определены «декорации» (оформление). Конечно же, под «декорированием» понимается не визуальное оформление, а добавление специфических функций. **Component** содержит объявление абстрактной операции, к конкретной реализации которой декоратором, впоследствии, будет добавлен «новый» функционал.
- **ConcreteComponent** — представляет собой реализацию компонента.
- **Decorator** — класс, который наследует и агрегирует компонент. Он переопределяет реализацию операции таким образом, чтобы выполнить функцию, инкапсулированную в компоненте, а затем добавить новый функционал.

Поскольку декоратор является частным случаем компонента (наследует **Component**), то в качестве инкапсули-

рованного компонента может быть использован другой декоратор, что позволяет осуществлять рекурсивную последовательность вызовов переопределённой операции с постепенным «накоплением» функционала (как показано выше на диаграмме последовательности).

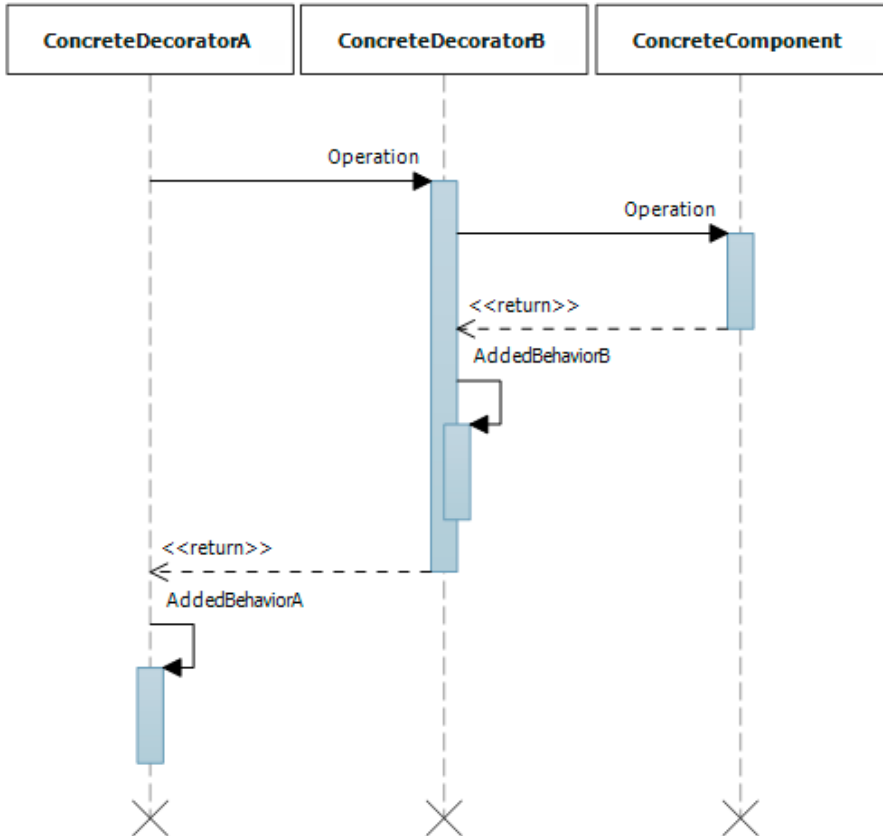


Рис. 5.3.1.

ConcreteDecoratorA и ConcreteDecoratorB — представляют собой частные реализации декоратора для компонента.

Структура паттерна проектирования декоратор (Decorator) иллюстрируется представленной ниже диаграммой классов.

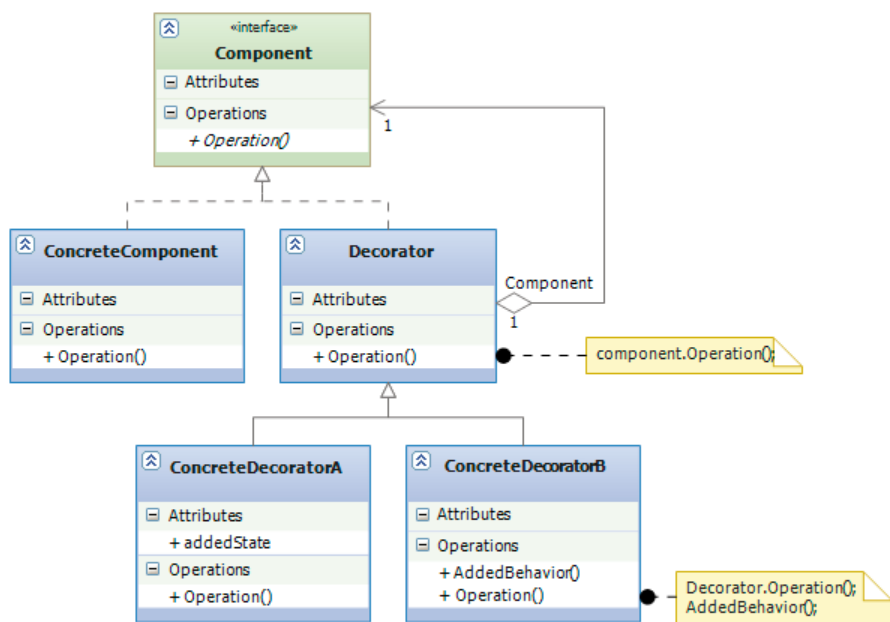


Рис. 5.3.2.

5.4. Результаты использования паттерна

Использование паттерна, как уже выше упоминалось, позволяет более гибко, нежели при использовании наследования реализовать распределение обязанностей по выполнению некоторой сложной задачи между несколькими классами.

С другой стороны, паттерн проектирования декоратор предотвращает перенасыщение иерархии классов, поскольку позволяет избежать необходимости создавать

классы, которые бы сочетали в себе функционал во всех необходимых комбинациях.

5.5. Практический пример использования паттерна

В качестве примера мы рассмотрим модель текстового редактора с подсветкой html-синтаксиса.

Предполагается использовать класс текстового представления, для реализации логики представления текстовой информации пользователю. Модель реализации текстового представления приведена ниже.

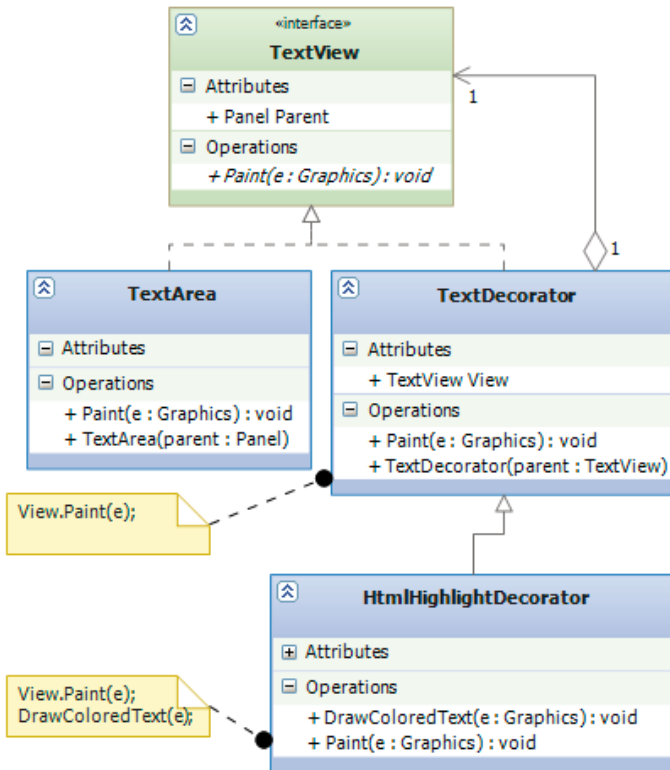


Рис. 5.5.3.

Класс текстового представления (`TextView`) наследуется, с одной стороны, классом конкретной реализации представления в виде текстовой области (класс `TextArea`), поддерживающей редактирование. Для поддержки возможности расширения функционала текстового представления, нами объявляется класс `TextDecorator`, который, собственно, реализует паттерн `Decorator`. Для реализации возможности подсветки html-синтаксиса нами создаётся конкретная реализация класса `TextDecorator`, которая инкапсулирует логику осуществления подсветки html-синтаксиса в текстовом представлении.

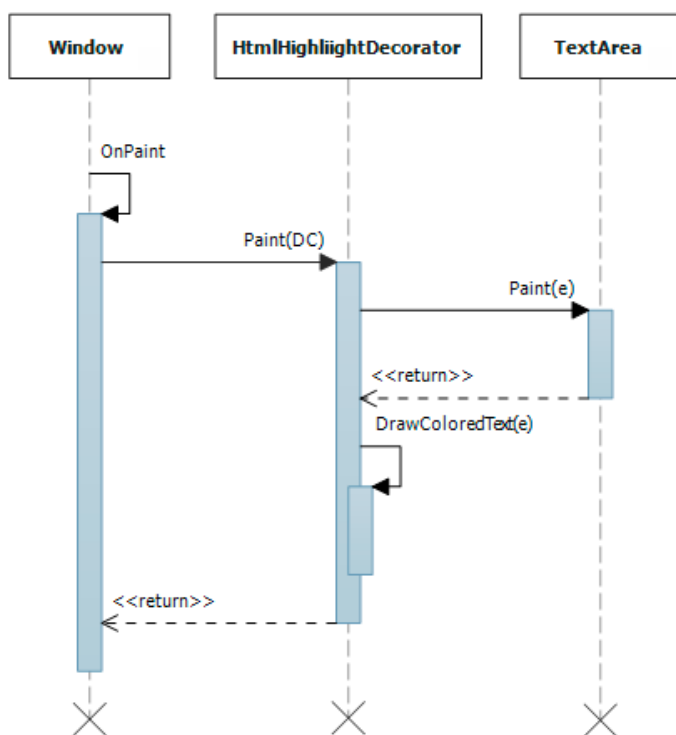


Рис. 5.5.4.

При осуществлении прорисовки окна, в котором используется текстовое представление, (обработчик события `Paint`) создаётся изображение по размеру клиентской области приложения, для которого инициализируется графический контекст. Объект графического контекста созданного изображения передаётся в метод `Paint` объекта класса `HtmlHighlightDecorator`, который вызывает метод `Paint` объекта класса `TextArea`, а впоследствии и метод `DrawColoredText`, который отвечает за прорисовку подсвеченного текста. Последовательность вызовов иллюстрируется на диаграмме последовательности, приведённой на рис. 5.5.4.

```

4      <component name=
      "Microsoft-Windows-International-Core-WinPE"
      processorArchitecture="x86" publicKeyToken=
      "31bf3856ad364e35" language="neutral" versionScope=
      "nonSxS" xmlns:wcm=
      "http://schemas.microsoft.com/WMIConfig/2002/State"
      xmlns:xsi=
      "http://www.w3.org/2001/XMLSchema-instance">
5          <SetupUILanguage>
6              <UILanguage>ru-RU</UILanguage>
7          </SetupUILanguage>
8          <InputLocale>en-US; ru-RU</InputLocale>
9          <SystemLocale>ru-RU</SystemLocale>
10         <UILanguage>ru-RU</UILanguage>
11         <UserLocale>ru-RU</UserLocale>
12     </component>

```

Рис. 5.5.5.

Таким образом, после возвращения управления в обработчик события `Paint` оконного класса, на созданном в обработчике изображении будет прорисован текст с подсвеченным синтаксисом. Последним действием мы прорисовываем полученное изображение на гра-

фическом контексте окна, с которым связано текстовое представление.

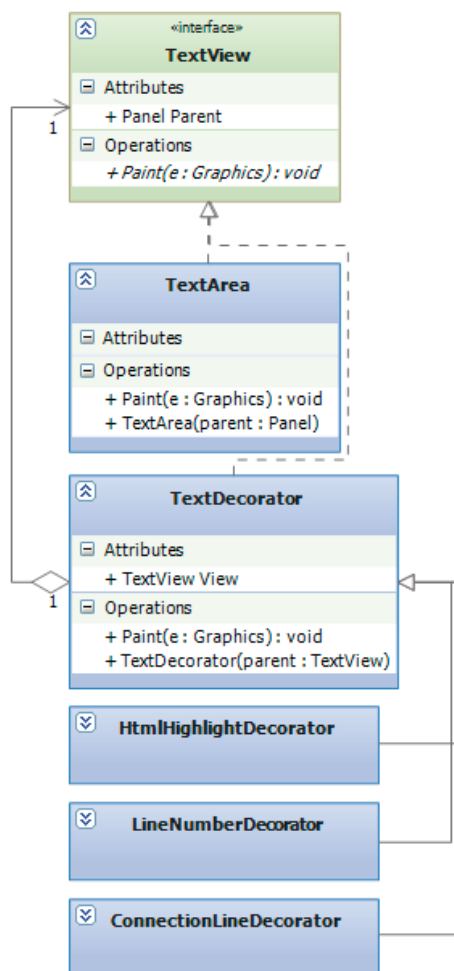


Рис. 5.5.6.

Подобная реализация позволит впоследствии гибко добавлять и убирать различные декорирующие эффекты, свойственные текстовым редакторам. Например, номера

строк, соединяющие линии для парных элементов, как показано на представленном выше рисунке.

Для этого достаточно объявить несколько декораторов, и добавить их для текстового представления в оконном классе, с которым связано текстовое представление.

Обобщённая модель приведена слева.

К уроку приложена папка с примерами реализации паттернов. Структурные паттерны расположены в папке Structural. Реализация паттерна Decorator находится в папке Decorator.

6. Паттерн Facade

Паттерн **Facade** (*фасад*) предназначен в большей мере для инкапсуляции (сокрытия) содержимого и разделения логических частей на независимые подсистемы. В «реальном» мире много, где применяется структура паттерна фасад, хорошим примером служит любой ресторан быстрого питания, где вы просто подходите к кассе и заказываете еду, а дальнейшая структура забегаловки вас, как правило, не интересует. Но в забегаловке также есть офис, в который вы тоже можете прийти, и обратиться к нему по «узкому» интерфейсу (например, стать постоянным клиентом). Таким образом, ресторан получает две минимально зависящие друг от друга подсистемы, которые вместе складываются в одну большую систему.

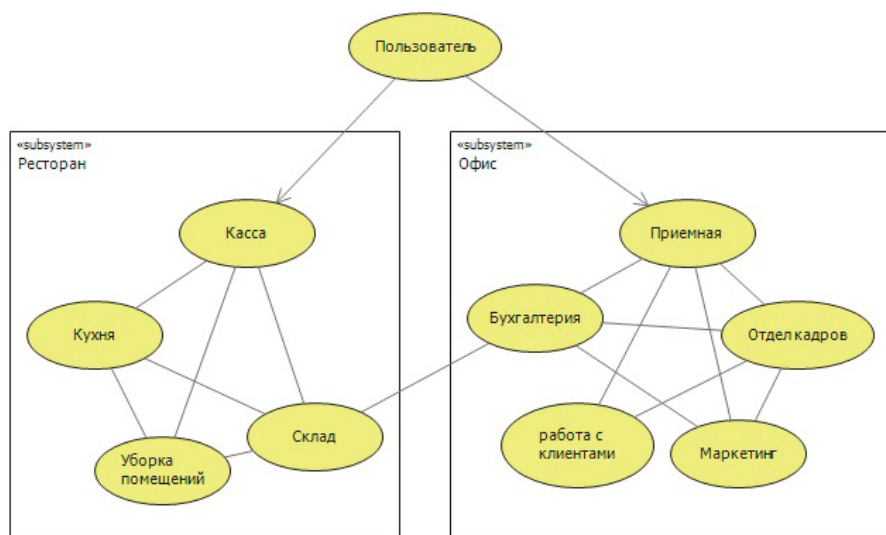


Рис. 6.5.1.

6.1. Цель паттерна

Целью паттерна фасад является определенная структуризация классов, в подсистемы, доступ к каждой из них происходит через один интерфейс. Так же при построении архитектуры приложения желательно его разделять на отдельные под системы, которые должны быть максимально независимы.

Каждая часть программы (подсистема) будет иметь свой класс фасад, через который будет производиться управление этой частью программы.

6.2. Причины возникновения паттерна

Спроектировать и потом построить огромное приложение «сразу» очень сложно, архитекторы решили эту задачу разбиением программы на более мелкие подсистемы, это позволяет строить какую-то одну часть программы, не отвлекаясь на другие ее аспекты, после построения переходить к написанию следующей подсистемы. А если строить системы не зависимыми друг от друга можно получить «безопасную» заменимость ее компонентов или целых подсистем, также, получая независимую подсистему, вы можете использовать ее повторно в других проектах, и наконец если в команде программистов более одного человека это позволит поручить каждому(или нескольким) из них написать отдельную подсистему, а потом просто собрать из в целое приложение.

Для того, чтобы воспользоваться незнакомой вам подсистемой приходилось перечитывать немало документации об этой структуре, и разбираться как ней

пользоваться. Для решения этой проблемы, был создан паттерн **Facade** (*Фасад*). Который подразумевает, что для каждой подсистемы, будет создан свой класс, который является своего рода хранителем подсистемы. Через такой класс будет легко взаимодействовать со структурой.

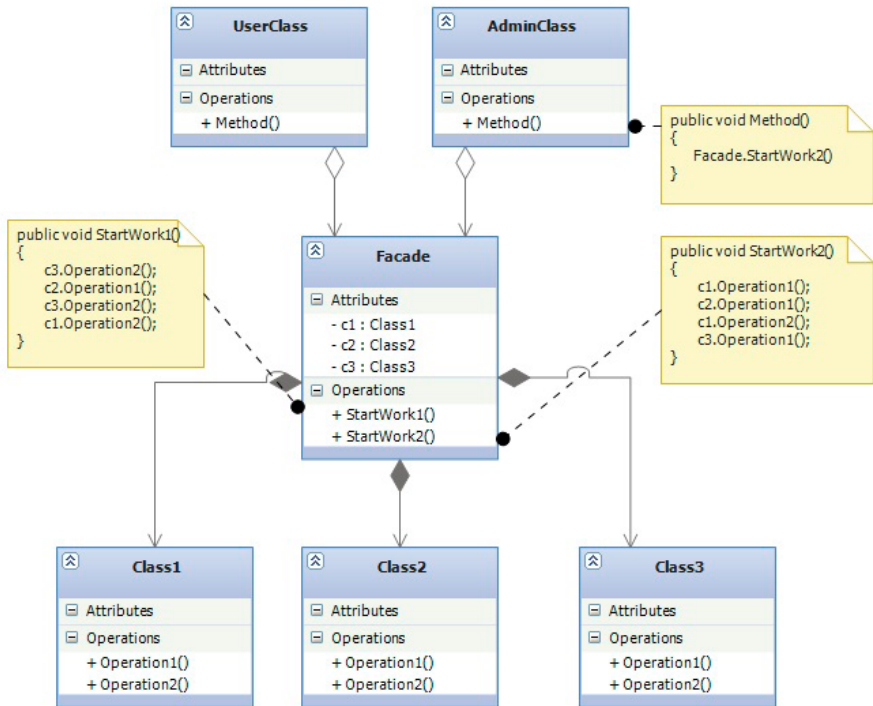


Рис. 6.2.2.

6.3. Структура паттерна

Паттерн фасад подразумевает существование некоторой подсистемы в программе, для которой создается класс фасада. Через класс фасада иницируется выполнение операций из классов подсистемы.

Стоит так же отметить то, что клиенты (пользователи подсистемы) не должны иметь доступа к классам подсистемы.

Все объекты подсистемы, если это возможно должны храниться в классе фасад. При вызове клиентами метода из объекта **Facade**, он начинает работу с классами подсистемы.

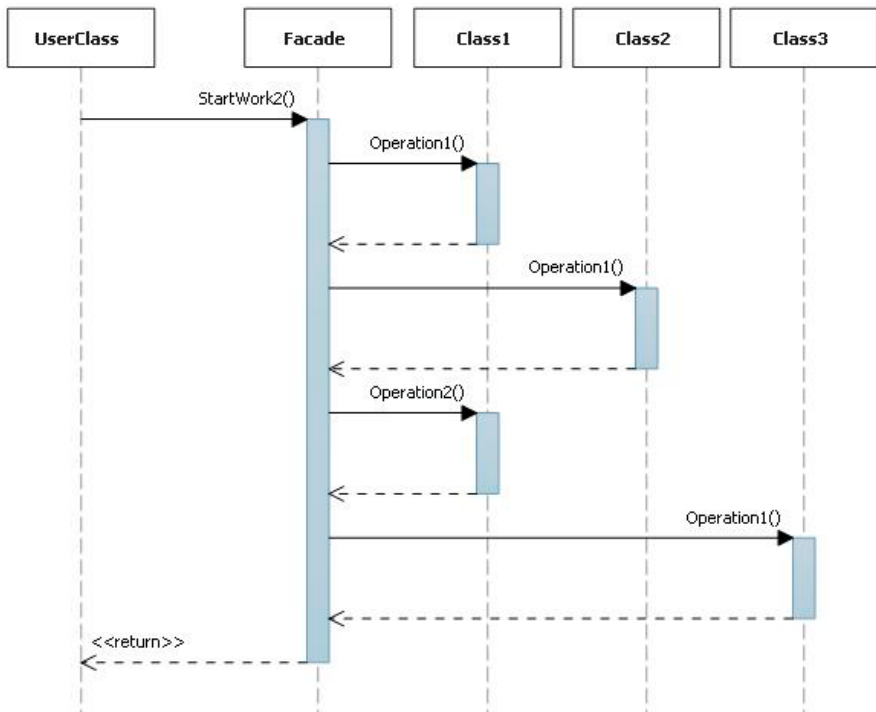


Рис. 6.3.3.

Описанный выше пример, не является правилом. В паттерне фасад могут иметься более или менее чем три класса внутри подсистемы. Объект класса фасад может не только вызывать методы из элементов подсистемы, он

может их настраивать, присваивать их свойствам значения и так далее.

Необязательно класс фасада должен хранить в себе объекты классов подсистемы.

Доступ к объектам подсистемы можно обустроить через паттерн **Proxy**. Также иногда программисты делают класс фасада статическим для того, чтобы к нему можно было обратиться с любой точки программы.

6.4. Результаты использования паттерна

Строить программу, начиная с написания отдельных независимых подсистем проще, чем писать сразу всё приложение. Вдобавок, мы получаем готовый блок программы, который можно использовать и в других приложениях. Также можно произвести замену одного из компонентов подсистемы, не нарушая общей структуры приложения.

Если для каждой подсистемы создавать свой класс фасада, пользоваться такой системой будет проще, так как вся работа этой структуры инкапсулирована (спрятана) в классе фасад.

Используя другие паттерны вместе с паттерном **Facade** можно достичь большей производительности, гибкости, безопасности приложения.

6.5. Практический пример

Рассмотрим паттерн фасад в примере следующего приложения: простой переводчик слов, пользователь вводит слово на английском и в результате получает его перевод на русском.

Приложение должно предоставлять возможность работать со словарем заранее записанных слов (сохранять, загружать) с файла. Также приложение должно уметь добавлять новые слова в текстовую базу данных.

В программе предусмотрена подсистема для работы со словарем, доступ к которой будет осуществляться через объект класса фасад.

Далее представлена диаграмма классов этого приложения.

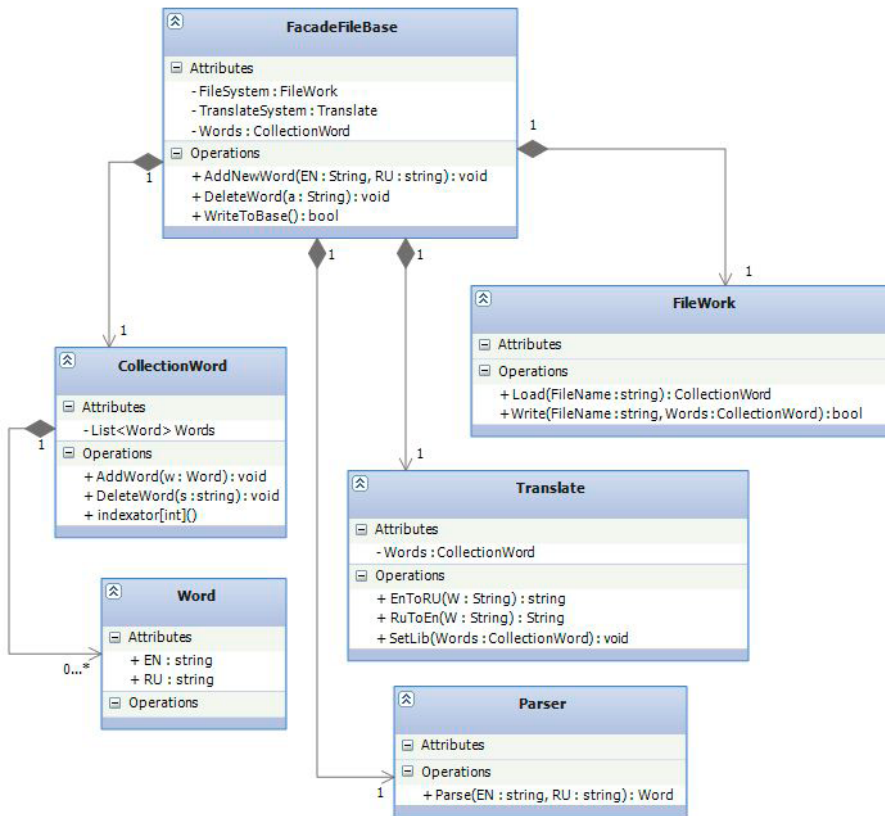


Рис. 6.5.4.

Пользователь будет управлять системой через класс `FacadeFileBase` (фасад) который в свою очередь вызывает объекты подсистемы.

К уроку приложена папка с примерами реализации паттернов. Структурные паттерны расположены в папке `Structural`. Реализация паттерна `Facade` находится в папке `Facade`.

7. Паттерн Flyweight

7.1. Цель паттерна

Целью паттерна является более экономное использование памяти компьютера, достигается это более правильным способом работы с большим количеством мелких объектов.

Для понимания паттерна необходимо научиться разделять внутренние и внешние свойства объекта. Внутреннее свойство объекта — это свойство, которое хранит объект внутри себя, другими словами, это экземпляр класса внутри которого создано свойство (переменная) и храниться до тех пор, пока «живет» объект. Внешнее свойство — это свойство, которое передается объекту как аргумент одного (или нескольких) метода, и существует, пока выполняется метод.

Суть паттерна заключается в том, чтобы «переписать» внутренние свойства во внешние, и после не создавать много объектов, а создать один который будет «отображаться» по-разному, в зависимости от того какие внешние свойства к нему будут применены.

7.2. Причины возникновения паттерна

После «перехода» на ООП стало проще программировать так как все объекты «как настоящие», но это сопряжено с некоторым количеством трудностей. Создавая модель приложения, хочется расположить объекты по принципам объектно-ориентированного программирования, например стул состоит из ножек и сидения, а сидение состоит еще из нескольких деталей. Если описывать такую

архитектуру слишком «глубоко» может возникнуть ситуация, когда объектов будет слишком много, от чего приложение с такой архитектурой будет «тормозить». Что же делать, когда необходимый уровень «глубины» недостижим для компьютера? Ответ: разделить логическое понимание приложение от физического, вить в отличии от реальной жизни в программировании можно встретить моменты, когда одна и та же сущность находится «рисуеться» в двух (или более) местах. Так например логически у нас есть стул, состоящий из ножек и сидения, а физически в программе создан объект ножки, которая фигурирует в разных местах. Для того, чтобы объект фигурировал в разных местах достаточно просто при вызове из объекта метода «отобразить» передавать в него, некоторое количество параметров, которые и будут указывать, где и как отображать этот элемент. Используя такой подход к построению архитектуры программист получит приложение, которое будет достаточно экономно использовать ресурсы компьютера.

7.3. Структура паттерна

Объект, который «фигурирует» в нескольких местах одновременно называется приспособленцем. Доступ к таким объектам предоставляется посредством фабрики. Клиент создает объекты приспособленцев с помощью фабрики, вызывая метод, который возвращает объект приспособленца и принимает идентификатор объекта, который ему необходимо вернуть.

Фабрика хранит в себе (коллекцию, массив, отдельно) объектов приспособленцев. Важно, что объекты должны

создаваться при первом запросе клиента, а при последующих запросах им должен возвращаться уже созданный объект, который как было сказано выше должен храниться внутри фабрики.

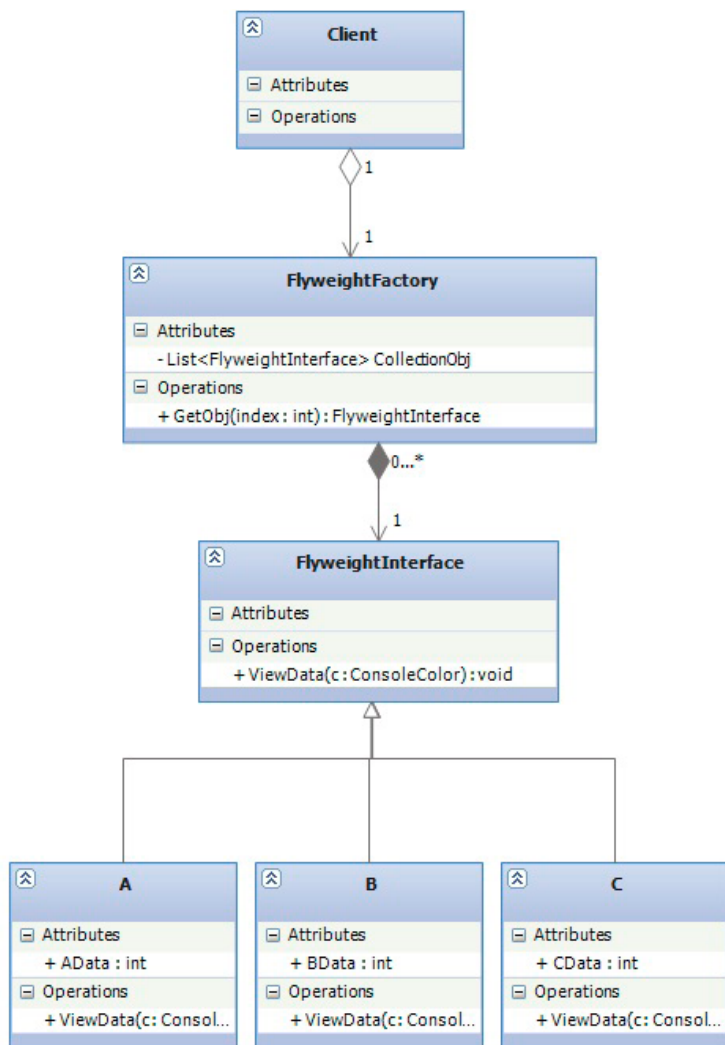


Рис. 7.3.1.

При вызове метода `GetObj` пользователь передает в него идентификатор объекта, который ему необходимо получить, если объект с таким индексом уже создан, его необходимо вернуть как результат работы метода, но если объект не создан, его необходимо создать, и записать в хранилище фабрики, после чего вернуть как его как результат работы метода.

Классы А, В, С наследуются от базового класса, который описывает интерфейс взаимодействия со всеми типами приспособленцев.

7.4. Результаты использования паттерна

Если применять этот паттерн к своему приложению, вы получите действительно сильную экономию памяти, даже учитывая то, что при получении доступа к объекту будет теряться время в методе фабрики (в нашем случае `GetObj`). Но экономия памяти получится лишь в том случае, если в объектах по минимуму будут содержаться внутренние свойства, а при вызове будут передаваться внешние. Внешние свойства должны вычисляться алгоритмами, в этом и состоит основная идея паттерна, «экономить память за счет вычислительной мощности».

Получать доступ к объектам приспособленцев стоит только через методы фабрики, так как это добавит гибкости и возможности настроить объект в методах фабрики.

Применять паттерн следует тогда, когда в вашем приложении действительно большое количество объектов, и когда их состояние можно описать внешними свойствами, и внешние свойства можно описать и присвоить с помощью вычислений.

7.5. Практический пример

Рассмотрим консольное приложение, которое будет отображать, и просчитывать итог сражения двух армий.

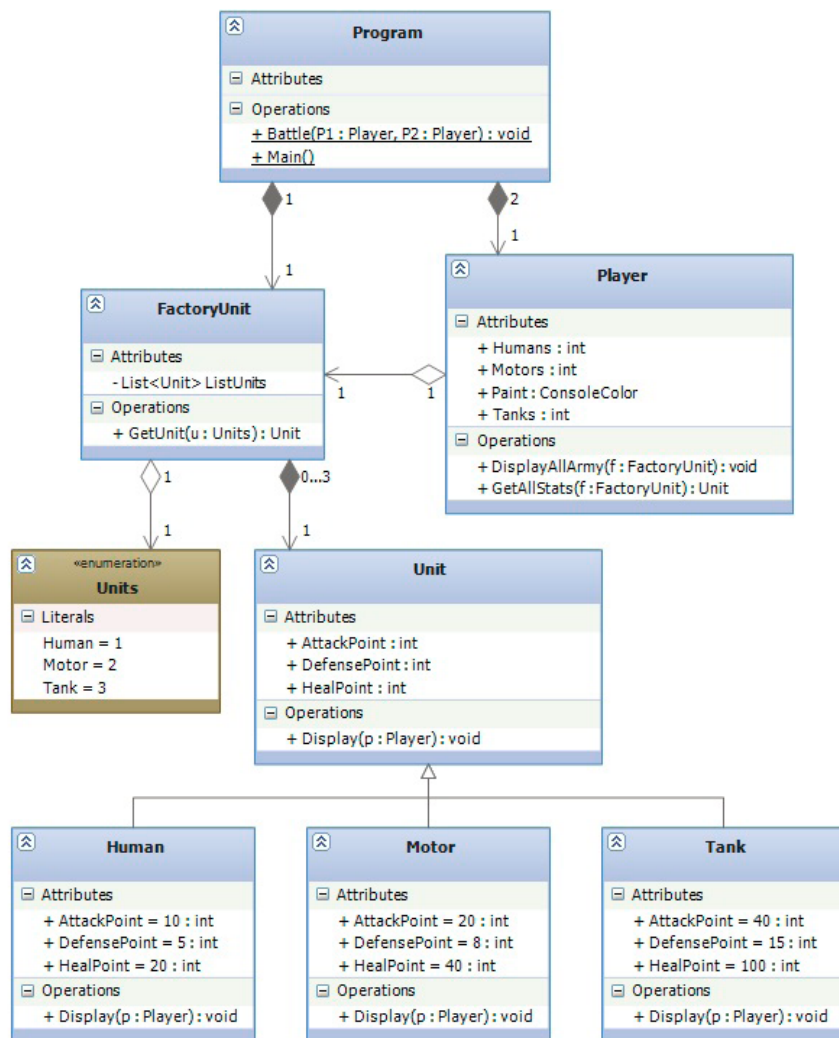


Рис. 7.5.2.

Классы `Human`, `Motor` и `Tank`, отличаются друг от друга только значениями внутренних свойств, и методом, отображающим на экране боевую единицу.

Объект класса `FactoryUnit` будет хранить в себе от нуля до трех объектов в виде `Unit` в специально для этого созданной коллекции `ListUnits`. Классы клиентов смогут получить доступ, к объекту вызвав из фабрики метод и передав в него объект перечисления, который и укажет какой конкретно `Unit` надо вернуть как результат вызова метода.

Класс игрока (`Player`) будет использовать фабрику для получения и отображения свойств объектов. Но пользоваться все объекты игроков будут на самом деле только 3-мя экземплярами классов потомков `Unit` это и должно экономить память.

Основной класс `Program` содержит метод `Main`, в котором происходит управление всей программой, а также метод `Battle` отвечающий за математику, которая покажет какой был исход битвы армий двух игроков.

К уроку приложена папка с примерами реализации паттернов. Структурные паттерны расположены в папке `Structural`. Реализация паттерна `Flyweight` находится в папке `Flyweight`.

8. Паттерн Proxy

Паттерн **Proxy** часто именуют, как паттерн суррогат. Далее основной класс, о котором будет идти речь, именуется суррогатом, или прокси, оба термина являются верными и описывают один и тот же класс/объект.

Прокси это — некоторый объект, обеспечивающий транзитный доступ к другому объекту.

Суррогат это — некоторый объект, который внешне ничем не отличается от основного, но внутренней функциональности в нем нет.

Объект Суррогата или прокси внешне ничем не отличается от объекта, который он представляет, но вся его внутренняя функциональность лишь является некоторым туннелем, через который классы клиента получают доступ к основным объектам.

Так же объект суррогата или прокси можно называть заместителем объекта.

8.1. Цель паттерна

Целью паттерна является создание системы доступа к объекту через специальный объект суррогата.



Рис. 8.1.1.

Это позволяет добиться большей гибкости работы с объектом целевого класса.

Если объект целевого класса после создания будет занимать много места и нет гарантии того, что объект будет необходим в работе приложения, можно создавать целевой объект только когда клиент первый раз вызывает метод из объекта суррогата это ускорит быстроедействие приложения в случае, если объект не будет использоваться. А если объект и будет создан, то на этапе выполнения программы это будет не так «болезненно» для пользователя.

Такая система доступа позволит так же использовать целевой класс с большей защитой (в суррогате можно выполнять различные проверки принимаемых параметров). Например, если целевой класс — это калькулятор, просто принимает в аргументы метода два параметра и выполняет базовые математические операции, в суррогате класса калькулятор можно выполнить проверку чтобы не получилось, что калькулятору будет передана задача, «поделить на ноль».

Можно, использовать структуру паттерна прокси для создания так называемого «Посла» в другое пространство имен. Например, целевой класс описан в пространстве имен, которое по каким-то причинам невозможно (или неудобно) подключить, в таком случае можно создать суррогат, который будет находиться в «нужном» пространстве имен, а внутри суррогата будет создан объект целевого класса из другого пространства имен.

8.2. Причины возникновения паттерна

Код одних программистов должен быть понятен другим программистам, касательно как синтаксических «традиций», так и структурных. Если строить приложе-

ние, используя паттерн прокси для доступа к объектам и в объектах суррогатов заниматься проверкой данных, а в самом целевом классе выполнять логику. Потом достаточно легко объяснить другому программисту, где выполняются проверки, а где основная логика. Вообще используя паттерны довольно легко объяснить структуру приложения.

Наверное, вам уже приходилось видеть приложения, которые после начала запуска, очень долго «думают» и только по истечении 5 или более секунд начинают отображать свой интерфейс. Так получается, потому что приложение сразу при старте начинает загружать все свои модули, создает все экземпляры классов, которые понадобятся (или не понадобятся) в работе приложения. Для частичного решения такой проблемы, можно создать суррогат «тяжелого» объекта. Создавать такой объект при старте приложения, суррогат это лишь муляж который не является таким «тяжелым» как целевой объект и потому создается быстрее, а «тяжелый» объект будет создан только, тогда, когда он будет действительно необходим. А необходимость появляется, когда классы клиента будут работать с объектом суррогата, который все запросы будет перенаправлять на только что созданный целевой объект.

8.3. Структура паттерна

Паттерн **Proxy** подразумевает наличие некоторого объекта (далее «целевой класс»), к которому будет осуществляться доступ с помощью специально созданного объекта суррогата.

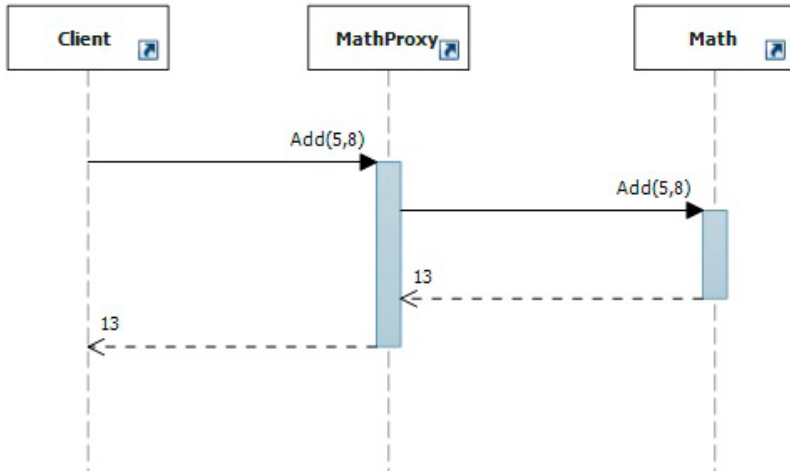


Рис. 8.3.2.

Но объект суррогата не должен отличаться внешне от объекта целевого класса, потому что оба класса должны быть унаследованы от одного интерфейса.

В приведённом далее примере целевой объект `Math` создается сразу с объектом суррогата, так как класс `math` не является «тяжелым» и будет создаваться быстро.

Пример показывает, как с помощью объекта суррогата можно выполнять различные проверки перед непосредственной передачей данных целевому классу.

8.4. Результаты использования паттерна

Реализовав доступ к паттерну через объект суррогата, мы получаем дополнительный уровень работы с целевым объектом. На этом уровне мы можем заниматься оптимизацией, создавая целевые объекты только по мере того, как они будут использоваться. Можно заниматься проверкой данных перед передачей в целевой объект.

В объекте суррогата можно даже вести логирование доступа к объекту (например, высчитывать сколько раз был вызван метод суммирования на калькуляторе).

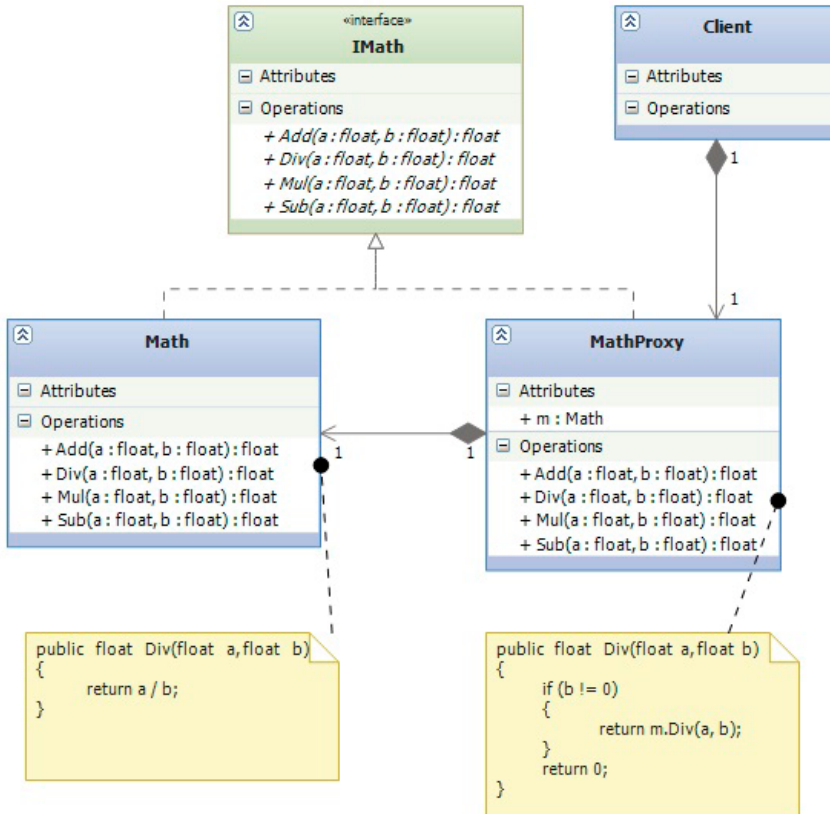


Рис. 8.4.3.

8.5. Практический пример

Для лучшего понимания паттерна ознакомимся со структурой приложения «магазин информации». В архитектуре приложения имеется класс **Shop** к которому осуществляется доступ через класс **ProxyShop**. Класс **Shop**

имеет метод, который принимает строку с ключом, а потом если возможно возвращает строку с информацией.

Класс **ProxyShop** защищает класс **Shop** от возможности его «брутфорсить» также объект **Shop** создается только тогда, когда он будет необходим в программе, другими словами, тогда, когда кто-то попытается получить информацию по ключу.

Ниже представлена диаграмма классов этого приложения.

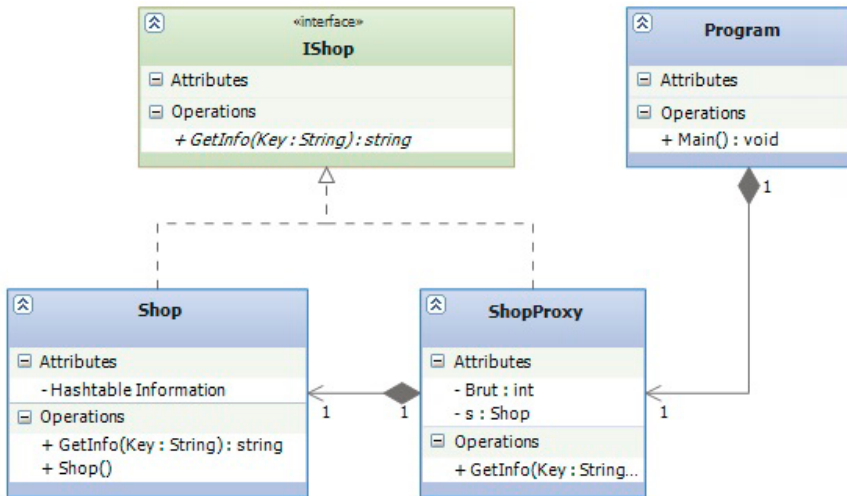


Рис. 8.5.4.

К уроку приложена папка с примерами реализации паттернов. Структурные паттерны расположены в папке *Structural*. Реализация паттерна *Proxy* находится в папке *Proxy*.

9. Анализ и сравнение структурных паттернов

Вы уже изучили все из структурных паттернов:

- **Adapter** — паттерн который позволяет «адаптировать» объект под другой интерфейс, для доступа к нему. Например в объекте имеется метод **Operation1**, а нам необходимо сделать так чтоб он назывался **OperationA**. Целью паттерна **Adapter** является «редактирование» интерфейса доступа к целевому объекту.
- **Bridge** — паттерн позволяет отделить абстракцию от реализации, когда имеется иерархия объектов которая описана абстрактно и позже будет реализоваться иерархия под конкретную систему. Целью паттерна **Bridge** является построение отдельно абстракции и реализации и предоставление клиенту абстракции с помощью, которой он сможет управлять реализацией.
- **Composite** — структурный паттерн который выстраивает объекты по типу дерева. Целью этого паттерна является построение древовидной структуры для хранения объектов.
- **Decorator** — паттерн позволяющий структурировать таким образом что несколько объектов отображаются как один. И количество внутренних объектов может изменяться «на лету».
- **Facade** — структурный паттерн который рассказывает как строить большие приложения из мелких не

зависимых подсистем. Целью паттерна является создание узкого и понятного интерфейса для работы с подсистемой.

- **Flyweight** — паттерн позволяющий экономить место в случае если в программе имеется множество мелких объектов и их состояние можно вынести во внешние свойства которые вычисляются алгоритмами.
- **Proxy** — подразумевает создание объекта суррогата для целевого объекта чем может обеспечить большую гибкость, экономию памяти, защиту.

Подходить к использованию паттернов нужно отталкиваясь от их назначения, потому что с первого взгляда может показаться что паттерны многим похожи, но так кажется только потому что у некоторых из них похожая структура. Но у каждого паттерна есть цель использования. Например, вам может показаться паттерны фасад и прокси, очень похожи, но предназначены они для разных целей, цель паттерна фасад, создать узкий интерфейс работы с подсистемой, а прокси позволяет защитить целевой объект.

Структурно паттерны **Composite** и **Decorator** тоже очень похожи, но необходимо отталкиваться от целей применения, так как паттерн **Composite** структурирует объекты, а паттерн **Decorator** позволяет создать отдельные виды функциональности и создать из них «один» объект.

Часто приходится видеть, как люди не видят разницы между паттерном фасад(**Facade**) и паттерном адаптер(**Adapter**) потому, что кажется, что цель у них одна и так же, это изменить или модифицировать интерфейс

доступа к объекту или объектам. Но адаптер позволяет работать со старым интерфейсом доступа к объекту и использовать новый, а паттерн **Façade** создает новый интерфейс, более целенаправленный.

Важно отметить тот факт, что паттерн — это не эталон структуры программы, на практике приходится слышать, «ну вот у нас здесь архитектура как в паттерне **Flyweight**, но в фабрике разные типы объектов возвращаются из разных методов».

10. Домашнее задание

В качестве домашнего задания необходимо разработать три модели приложений, каждое из которых должно реализовывать хотя бы один структурный паттерн проектирования, описанный в текущем уроке. Каждая модель должна быть представлена в виде одной диаграммы классов, отражающей структуру классов, используемых этой моделью, а также связи, в которых состоят эти типы данных.

Одну из созданных моделей (на выбор) необходимо реализовать в виде работающего приложения.

Урок № 3
Паттерны поведения

1. Понятие паттерна поведения

Паттерны поведения (поведенческие паттерны), как видно из названия служат для управления различными вариантами поведения системы объектов (классов). В этом уроке мы рассмотрим некоторые из данных паттернов. В проекте, который идет с уроком вы найдете код всех паттернов.

2. Паттерн Chain Of Responsibility

Данный паттерн предназначен для того, чтобы позволять объекту отправлять команду, не имея информации об объекте(-ах), получающих ее. Важно отметить, что команда передается группе объектов, которая часто является частью более крупной структуры.

Каждый объект цепочки может обрабатывать, передавать полученную команду следующему объекту в цепи или делать и то, и другое.

Рассмотрим UML диаграмму для данного паттерна:

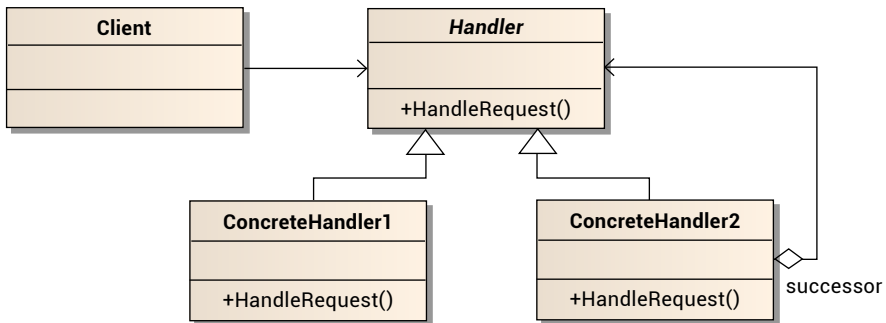


Рис. 2.5.1.

В данной диаграмме следующие участники:

- **Handler**
 - ▷ Определяет интерфейс для обработки запросов;
- **ConcreteHandler**
 - ▷ Обрабатывает запрос, предназначенный конкретному исполнителю;

- ▷ Если **ConcreteHandler** может обработать запрос он это делает, иначе запрос переадресовывается отправителю
- **Client (ChainApp)**
 - ▷ Иницирует запрос к объекту **ConcreteHandler**, находящемуся в цепочке

К уроку приложена папка с примерами реализации паттернов. Паттерны поведения расположены в папке Behavioral. Реализация паттерна Chain of Responsibility находится в папке Chain of Responsibility.

3. Паттерн Command

Паттерн **Command** инкапсулирует команды в некотором объекте. Инкапсулирование, таким образом, позволяет выполнять различные манипуляции, например такие как: управление выбором и последовательностью исполнения команд, возможность постановки команд в очередь, отмена команд и т.д.

Рассмотрим UML диаграмму для данного паттерна

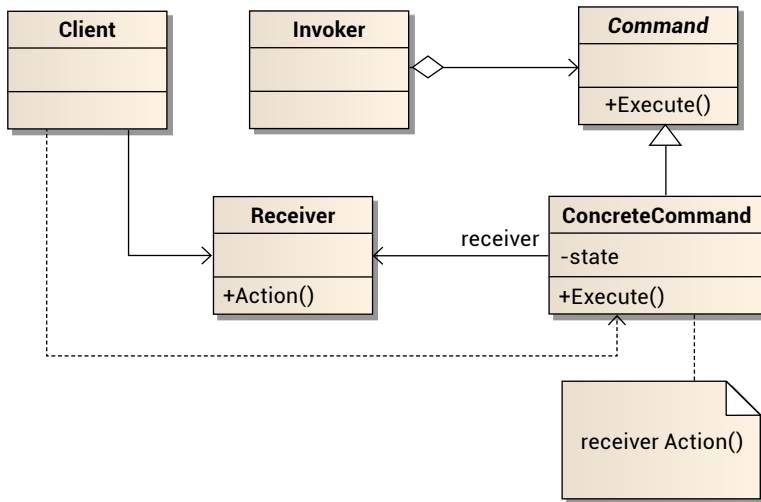


Рис. 3.5.1.

В данной диаграмме следующие участники:

- **Command**
 - ▷ Определяет интерфейс для исполнения операции.
- **ConcreteCommand**
 - ▷ Определяет связывание между объектом-получателем (**Receiver**) и действием;

- ▷ Реализует исполнение путем вызова соответствующих операций **Receiver**.
- **Client**
 - ▷ Создает объект **ConcreteCommand** и устанавливает его получателя.
- **Invoker**
 - ▷ Запрашивает команду выполнить некоторый запрос.
- **Receiver**
 - ▷ Знает, как выполнить операции, связанные с обработкой запроса.

К уроку приложена папка с примерами реализации паттернов. Паттерны поведения расположены в папке Behavioral. Реализация паттерна Command находится в папке Command.

4. Паттерн State

Паттерн **State** заключает состояния объекта в отдельные объекты, каждый из которых расширяет общий суперкласс.

Рассмотрим UML диаграмму для данного паттерна

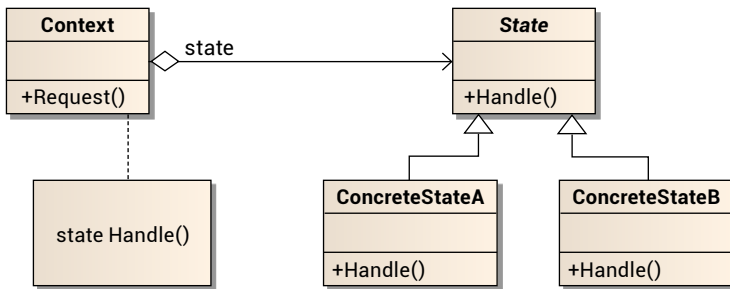


Рис. 4.5.1.

В данной диаграмме следующие участники:

- **Context**
 - ▷ Определяет интерфейс для клиентов;
 - ▷ Поддерживает объект наследника **ConcreteState**, определяющего текущее состояние.
- **State**
 - ▷ Определяет интерфейс для инкапсуляции поведения, связанного с состоянием **Context**.
- **ConcreteState**
 - ▷ Каждый наследник реализует поведение, связанное с состоянием **Context**.

К уроку приложена папка с примерами реализации паттернов. Паттерны поведения расположены в папке Behavioral. Реализация паттерна State находится в папке State.

5. Паттерн Template Method

Паттерн **Template Method** строится на абстрактном классе, содержащем часть логики, требуемой для исполнения задачи. Оставшаяся часть логики содержится в методах классов-потомков, которые создают свою реализацию абстрактных методов.

Рассмотрим UML диаграмму для данного паттерна:

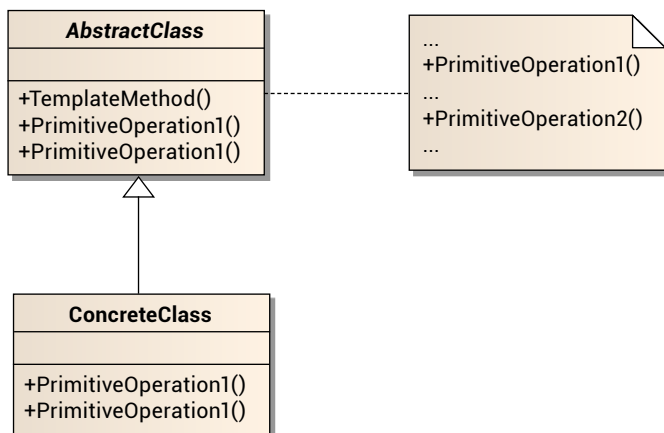


Рис. 5.5.1.

В данной диаграмме следующие участники:

■ AbstractClass

- Определяет абстрактные примитивные операции, которые будут определены потомками для реализации шагов алгоритма
- Реализует шаблонный метод, определяя скелет алгоритма. Шаблонный метод вызывает примитив-

ные операции, определенные в `AbstractClass` или в других объектах

- **ConcreteClass**

- ▷ Реализует примитивные операции, необходимые классам-потомкам для реализации алгоритмов

К уроку приложена папка с примерами реализации паттернов. Паттерны поведения расположены в папке Behavioral. Реализация паттерна Template Method находится в папке Template Method.

6. Паттерн Mediator

Паттерн **Mediator** используется для согласования изменений состояний набора объектов с помощью одного объекта. То есть вместо раскидывания логики поведения по разным классам данный паттерн инкапсулирует логику управления изменением состояний в рамки одного класса.

Рассмотрим UML диаграмму для данного паттерна:

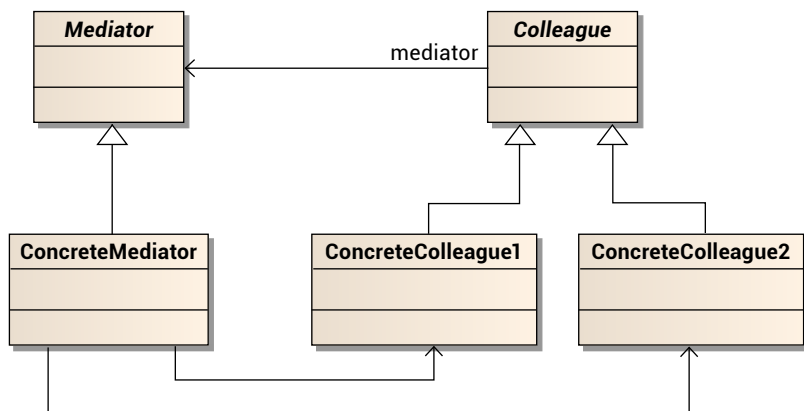


Рис. 6.5.1.

В данной диаграмме следующие участники:

- **Mediator**
 - ▷ Определяет интерфейс для общения с объектами **Colleague**.
- **ConcreteMediator**
 - ▷ Реализует совместное поведение путем координирования объектов **Colleague**;
 - ▷ Знает и поддерживает своих **Colleague**.

- **Colleague классы**

- ▷ Каждый **Colleague** класс знает своего **Mediator**;
- ▷ Каждый **Colleague** общается со своим медиатором.

К уроку приложена папка с примерами реализации паттернов. Паттерны поведения расположены в папке Behavioral. Реализация паттерна Mediator находится в папке Mediator.

В данном уроке мы привели часть паттернов поведения. Оставшиеся паттерны вам предназначены для самостоятельной проработки. Код всех паттернов поведения доступен для вас в папке **Behavioral**.

7. Экзаменационное задание

Реализуйте с использованием паттернов проектирования простейшую систему планирования задач. Должна быть возможность создания списка дел, установки приоритетов, установки дат выполнения, удаление и изменения дел. Каждому делу можно установить тег. Список дел можно загружать и сохранять в файл. Необходимо реализовать возможность поиска конкретного дела. Критерии поиска: по датам, по тегам, по приоритету и так далее.

